Fundamentos de JavaScript

Aprendamos los fundamentos para construir código.

1. ¡Hola, mundo!
2. Estructura del código
3. El modo moderno, "use strict"
4. Variables
5. Tipos de datos
6. Interacción: alert, prompt, confirm
7. Conversiones de Tipos
8. Operadores básicos, matemáticas
9. Comparaciones
10. Ejecución condicional: if, '?'
11. Operadores Lógicos
12. Operador Nullish Coalescing '??'
13. Bucles: while y for
14. La sentencia "switch"
15. Funciones
16. Expresiones de función
17. Funciones Flecha, lo básico
18. Especiales JavaScript

© 2007—2026 Ilya Kantor

acerca del proyecto
contáctenos

El Tutorial de JavaScript Moderno

Última actualización el 28 de octubre de 2026

Cómo se hace ahora. Desde lo básico hasta temas avanzados con explicaciones simples pero detalladas.

Buscar en el tutorial

Buscar

Comprar EPUB/PDF

Compartir:

627 ★ github

Discord Chat (English)

Tabla de contenidos

El curso principal contiene 2 partes que cubren JavaScript como lenguaje de programación y el trabajo con un navegador. También hay series adicionales de artículos temáticos.

PARTE 1
El lenguaje JavaScript

PARTE 2
El navegador: Documentos, Eventos e Interfaces

PARTE 3
Artículos adicionales

El lenguaje JavaScript

Aquí aprenderemos JavaScript, empezando desde cero y llegaremos hasta conceptos avanzados como POO.

Nos concentraremos en el lenguaje mismo con el mínimo de notas específicas del entorno.

Una introducción

1.1 Una introducción a JavaScript
1.2 Manuales y especificaciones
1.3 Editores de Código
1.4 Consola de desarrollador

Fundamentos de JavaScript

2.1 ¡Hola, mundo!
2.2 Estructura del código
2.3 El modo moderno, "use strict"
2.4 Variables
2.5 Tipos de datos
2.6 Interacción: alert, prompt, confirm
2.7 Conversiones de Tipos
2.8 Operadores básicos, matemáticas
2.9 Comparaciones
2.10 Ejecución condicional: if, '?'
2.11 Operadores Lógicos
2.12 Operador Nullish Coalescing '??'
2.13 Bucles: while y for
2.14 La sentencia "switch"
2.15 Funciones
2.16 Expresiones de función
2.17 Funciones Flecha, lo básico
2.18 Especiales JavaScript

Calidad del código

3.1 Debugging en el navegador
3.2 Estilo de codificación
3.3 Comentarios
3.4 Código ninja
3.5 Automatización de pruebas con Mocha
3.6 Polyfills y transpiladores

Objetos: lo básico

4.1 Objetos
4.2 Referencias de objetos y copia
4.3 Recolección de basura
4.4 Métodos de objeto, "this"
4.5 Constructor, operador "new"
4.6 Encadenamiento opcional '?.'
4.7 Tipo Symbol
4.8 Conversión de objeto a valor primitivo

Tipos de datos

5.1 Métodos en tipos primitivos
5.2 Números
5.3 Strings
5.4 Arrays
5.5 Métodos de arrays
5.6 Iterables
5.7 Map y Set
5.8 WeakMap y WeakSet
5.9 Object.keys, values, entries
5.10 Asignación desestructurante
5.11 Fecha y Hora
5.12 Métodos JSON, toJSON

Trabajo avanzado con funciones

6.1 Recursión y pila
6.2 Parámetros Rest y operador Spread
6.3 Ámbito de Variable y el concepto "closure"
6.4 La vieja "var"
6.5 Objeto Global
6.6 Función como objeto, NFE
6.7 La sintaxis "new Function"
6.8 Planificación: setTimeout y setInterval
6.9 Decoradores y redirecciones, call/apply
6.10 Función bind: vinculación de funciones
6.11 Funciones de flecha revisadas

Configuración de las propiedades de objetos

7.1 Indicadores y descriptores de propiedad
7.2 "Getters" y "setters" de propiedad

Prototipos y herencia

8.1 Herencia prototípica
8.2 F.prototype
8.3 Prototipos nativos
8.4 Métodos prototipo, objetos sin __proto__

Clases

9.1 Sintaxis básica de `class`
9.2 Herencia de clase
9.3 Propiedades y métodos estáticos.
9.4 Propiedades y métodos privados y protegidos.
9.5 Ampliación de clases nativas
9.6 Comprobación de clase: "instanceof"
9.7 Los Mixins

Manejo de errores

10.1 Manejo de errores, "try...catch"
10.2 Errores personalizados, extendiendo Error

Promesas y async/await

11.1 Introducción: callbacks
11.2 Promesa
11.3 Encadenamiento de promesas
11.4 Manejo de errores con promesas
11.5 Promise API
11.6 Promisificación
11.7 Microtareas (Microtasks)
11.8 Async/await

Generadores e iteración avanzada

12.1 Generadores
12.2 Iteradores y generadores asíncronos

Módulos

13.1 Módulos, introducción
13.2 Export e Import
13.3 Importaciones dinámicas

Temas diversos

14.1 Proxy y Reflect
14.2 Eval: ejecutando una cadena de código
14.3 Currificación
14.4 Tipo de Referencia
14.5 BigInt
14.6 Unicode, String internals
14.7 WeakRef y FinalizationRegistry

El navegador: Documentos, Eventos e Interfaces

Aprenderemos a manejar la página del navegador: agregar elementos, manipular su tamaño y posición, crear interfaces dinámicamente e interactuar con el visitante.

Documento

1.1 Entorno del navegador, especificaciones
1.2 Árbol del Modelo de Objetos del Documento (DOM)
1.3 Recorriendo el DOM
1.4 Buscar: getElement*, querySelector*
1.5 Propiedades del nodo: tipo, etiqueta y contenido
1.6 Atributos y propiedades
1.7 Modificando el documento
1.8 Estilos y clases
1.9 Tamaño de elementos y desplazamiento
1.10 Tamaño de ventana y desplazamiento
1.11 Coordenadas

Introducción a los eventos

2.1 Introducción a los eventos en el navegador
2.2 Propagación y captura
2.3 Delegación de eventos
2.4 Acciones predeterminadas del navegador
2.5 Envío de eventos personalizados

Eventos en la UI

3.1 Eventos del Mouse
3.2 Moviendo el mouse: mouseover/out, mouseenter/leave
3.3 Arrastrar y Soltar con eventos del ratón
3.4 Eventos de puntero
3.5 Teclado: keydown y keyup
3.6 Desplazamiento

Formularios y controles

4.1 Propiedades y Métodos de Formularios
4.2 Enfocado: enfoque/desenfoque
4.3 Eventos: change, input, cut, copy, paste
4.4 Formularios: evento y método submit

El documento y carga de recursos

5.1 Página: DOMContentLoaded, load, beforeunload, unload
5.2 Scripts: async, defer
5.3 Carga de recursos: onload y onerror

Temas diversos

6.1 Mutation observer
6.2 Selection y Range
6.3 Loop de eventos: microtareas y macrotareas

Artículos adicionales

Lista de temas adicionales que supone que ha cubierto las dos primeras partes del tutorial. Aquí no hay una jerarquía clara, puede leer los artículos en el orden que desee.

Marcos y ventanas

1.1 Ventanas emergentes y métodos de ventana
1.2 Comunicación entre ventanas
1.3 El ataque de secuestro de clics

Datos binarios y archivos

2.1 ArrayBuffer, arrays binarios
2.2 TextDecoder y TextEncoder
2.3 Blob
2.4 File y FileReader

Solicitudes de red

3.1 Fetch
3.2 FormData
3.3 Fetch: Progreso de la descarga
3.4 Fetch: Abort
3.5 Fetch: Cross-Origin Requests
3.6 Fetch API
3.7 Objetos URL
3.8 XMLHttpRequest
3.9 Carga de archivos reanudable
3.10 Sondeo largo
3.11 WebSocket
3.12 Eventos enviados por el servidor

Almacenando datos en el navegador

4.1 Cookies, document.cookie
4.2 LocalStorage, sessionStorage
4.3 IndexedDB

Animaciones

5.1 Curva de Bézier
5.2 Animaciones CSS
5.3 Animaciones JavaScript

Componentes Web

6.1 Desde la altura orbital
6.2 Elementos personalizados
6.3 Shadow DOM
6.4 Elemento template
6.5 Shadow DOM slots, composición
6.6 Estilo Shadow DOM
6.7 Shadow DOM y eventos

Expresiones Regulares

7.1 Patrones y banderas (flags)
7.2 Clases de caracteres
7.3 Unicode: bandera "u" y clase \p{...}
7.4 Anclas: inicio ^ y final $ de cadena
7.5 Modo multilínea de anclas ^ $, bandera "m"
7.6 Límite de palabra: \b
7.7 Escapando, caracteres especiales
7.8 Conjuntos y rangos [...]
7.9 Cuantificadores +, *, ? y {n}
7.10 Cuantificadores codiciosos y perezosos
7.11 Grupos de captura
7.12 Referencias inversas en patrones: \N y \k<nombre>
7.13 Alternancia (O) |
7.14 Lookahead y lookbehind (revisar delante/detrás)
7.15 Backtracking catastrófico
7.16 Indicador adhesivo “y”, buscando en una posición.
7.17 Métodos de RegExp y String

Comentarios

lea esto antes de comentar...



© 2007—2026 Ilya Kantor

acerca del proyecto
contáctenos

El lenguaje JavaScript

Aquí aprenderemos JavaScript, empezando desde cero y llegaremos hasta conceptos avanzados como POO.

Nos concentraremos en el lenguaje mismo con el mínimo de notas específicas del entorno.

1. Una introducción
2. Fundamentos de JavaScript
3. Calidad del código
4. Objetos: lo básico
5. Tipos de datos
6. Trabajo avanzado con funciones
7. Configuración de las propiedades de objetos
8. Prototipos y herencia
9. Clases
10. Manejo de errores
11. Promesas y async/await
12. Generadores e iteración avanzada
13. Módulos
14. Temas diversos

© 2007—2026 Ilya Kantor

28 de noviembre de 2021

¡Hola, mundo!

Esta parte del tutorial trata sobre el núcleo de JavaScript, el lenguaje en sí.

Pero necesitamos un entorno de trabajo para ejecutar nuestros scripts y, dado que este libro está en línea, el navegador es una buena opción. Mantendremos la cantidad de comandos específicos del navegador (como `alert`) al mínimo para que no pases tiempo en ellos si planeas concentrarte en otro entorno (como Node.js). Nos centraremos en JavaScript en el navegador en la siguiente parte del tutorial.

Primero, veamos cómo adjuntamos un script a una página web. Para entornos del lado del servidor (como Node.js), puedes ejecutar el script con un comando como `"node my.js"`.

La etiqueta “script”

Los programas de JavaScript se pueden insertar en casi cualquier parte de un documento HTML con el uso de la etiqueta `<script>`.

Por ejemplo:

```
<!DOCTYPE HTML>
<html>

<body>

  <p>Antes del script...</p>

  <script>
    alert( '¡Hola, mundo!' );
  </script>

  <p>...Después del script.</p>

</body>

</html>
```

Puedes ejecutar el ejemplo haciendo clic en el botón “Play” en la esquina superior derecha del cuadro de arriba.

La etiqueta `<script>` contiene código JavaScript que se ejecuta automáticamente cuando el navegador procesa la etiqueta.

Marcado moderno

La etiqueta `<script>` tiene algunos atributos que rara vez se usan en la actualidad, pero aún se pueden encontrar en código antiguo:

El atributo `type`: `<script type=…>`

El antiguo estándar HTML, HTML4, requería que un script tuviera un `type`. Por lo general, era `type="text/javascript"`. Ya no es necesario. Además, el estándar HTML moderno cambió totalmente el significado de este atributo. Ahora, se puede utilizar para módulos de JavaScript. Pero eso es un tema avanzado, hablaremos sobre módulos en otra parte del tutorial.

El atributo `language`: `<script language=…>`

Este atributo estaba destinado a mostrar el lenguaje del script. Este atributo ya no tiene sentido porque JavaScript es el lenguaje predeterminado. No hay necesidad de usarlo.

Comentarios antes y después de los scripts.

En libros y guías muy antiguos, puedes encontrar comentarios dentro de las etiquetas `<script>`, como el siguiente:

```
<script type="text/javascript"><!--
    ...
//--></script>
```

Este truco no se utiliza en JavaScript moderno. Estos comentarios ocultaban el código JavaScript de los navegadores antiguos que no sabían cómo procesar la etiqueta `<script>`. Dado que los navegadores lanzados en los últimos 15 años no tienen este problema, este tipo de comentario puede ayudarte a identificar códigos realmente antiguos.

Scripts externos

Si tenemos un montón de código JavaScript, podemos ponerlo en un archivo separado.

Los archivos de script se adjuntan a HTML con el atributo `src`:

```
<script src="/path/to/script.js"></script>
```

Aquí, `/path/to/script.js` es una ruta absoluta al archivo de script desde la raíz del sitio. También se puede proporcionar una ruta relativa desde la página actual. Por ejemplo, `src="script.js"` significaría un archivo `"script.js"` en la carpeta actual.

También podemos dar una URL completa. Por ejemplo:

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/lo
```

Para adjuntar varios scripts, usa varias etiquetas:

```
<script src="/js/script1.js"></script>
<script src="/js/script2.js"></script>
…
```

Por favor tome nota:

Como regla general, solo los scripts más simples se colocan en el HTML. Los más complejos residen en archivos separados.

La ventaja de un archivo separado es que el navegador lo descargará y lo almacenará en caché.

Otras páginas que hacen referencia al mismo script lo tomarán del caché en lugar de descargarlo, por lo que el archivo solo se descarga una vez.

Eso reduce el tráfico y hace que las páginas sean más rápidas.

Si se establece `src`, el contenido del script se ignora.

Una sola etiqueta `<script>` no puede tener el atributo `src` y código dentro.

Esto no funcionará:

```
<script src="file.js">
  alert(1); // el contenido se ignora porque se
</script>
```

Debemos elegir un `<script src="…">` externo o un `<script>` normal con código.

El ejemplo anterior se puede dividir en dos scripts para que funcione:

```
<script src="file.js"></script>
<script>
  alert(1);
</script>
```

Resumen

- Podemos usar una etiqueta `<script>` para agregar código JavaScript a una página.
- Los atributos `type` y `language` no son necesarios.
- Un script en un archivo externo se puede insertar con `<script src="path/to/script.js"> </script>`.

Hay mucho más que aprender sobre los scripts del navegador y su interacción con la página web. Pero tengamos en cuenta que esta parte del tutorial está dedicada al lenguaje JavaScript, por lo que no debemos distraernos con implementaciones específicas del navegador. Usaremos el navegador como una forma de ejecutar JavaScript, lo cual es muy conveniente para la lectura en línea, pero es solo una de muchas.

Tareas

Mostrar una alerta

importancia: 5

Crea una página que muestre el mensaje “¡Soy JavaScript!”.

Hazlo en un sandbox o en tu disco duro, no importa, solo asegúrate de que funcione.

Demo en nueva ventana

solución

Mostrar una alerta con un script externo

importancia: 5

Toma la solución de la tarea anterior Mostrar una alerta. Modificarla extrayendo el contenido del script a un archivo externo `alert.js`, ubicado en la misma carpeta.

Abrir la página, asegurarse que la alerta funcione.

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

20 de junio de 2021

Estructura del código

Lo primero que estudiaremos son los bloques de construcción del código.

Sentencias

Las sentencias son construcciones sintácticas y comandos que realizan acciones.

Ya hemos visto una sentencia, `alert('¡Hola mundo!')`, que muestra el mensaje "¡Hola mundo!".

Podemos tener tantas sentencias en nuestro código como queramos, las cuales se pueden separar con un punto y coma.

Por ejemplo, aquí separamos "Hello World" en dos alerts:

```
1 alert('Hola'); alert('Mundo');
```

Generalmente, las sentencias se escriben en líneas separadas para hacer que el código sea más legible:

```
1 alert('Hola');
2 alert('Mundo');
```

Punto y coma

Se puede omitir un punto y coma en la mayoría de los casos cuando existe un salto de línea.

Esto también funcionaría:

```
1 alert('Hola')
2 alert('Mundo')
```

Aquí, JavaScript interpreta el salto de línea como un punto y coma "implícito". Esto se denomina inserción automática de punto y coma.

En la mayoría de los casos, una nueva línea implica un punto y coma. Pero "en la mayoría de los casos" no significa "siempre"!

Hay casos en que una nueva línea no significa un punto y coma. Por ejemplo:

```
1 alert(3 +
2 1
3 + 2);
```

El código da como resultado `6` porque JavaScript no inserta punto y coma aquí. Es intuitivamente obvio que si la línea termina con un signo más `"+"`, es una "expresión incompleta", un punto y coma aquí sería incorrecto. Y en este caso eso funciona según lo previsto.

Pero hay situaciones en las que JavaScript "falla" al asumir un punto y coma donde realmente se necesita.

Los errores que ocurren en tales casos son bastante difíciles de encontrar y corregir.

Un ejemplo de error

Si tienes curiosidad por ver un ejemplo concreto de tal error, mira este código:

```
1 alert("Hello");
2
3 [1, 2].forEach(alert);
```

No es necesario pensar en el significado de los corchetes `[]` y `forEach` todavía, los estudiaremos más adelante. Por ahora, solo recuerda el resultado del código: muestra `Hello`, luego `1`, luego `2`.

Quitemos el punto y coma del alert:

```
1 alert("Hello")
2
3 [1, 2].forEach(alert);
```

La diferencia, comparando con el código anterior, es de solo un carácter: falta el punto y coma al final de la primera línea.

Esta vez, si ejecutamos el código, solo se ve el primer `Hello` (y un error pero necesitas abrir la consola para verlo). Los números no aparecen más.

Esto ocurre porque JavaScript no asume un punto y coma antes de los corchetes `[...]`, entonces el código del primer ejemplo se trata como una sola sentencia.

Así es como lo ve el motor:

```
1 alert("Hello")[1, 2].forEach(alert);
```

Se ve extraño, ¿verdad? Tal unión en este caso es simplemente incorrecta. Necesitamos poner un punto y coma después del `alert` para que el código funcione bien.

Esto puede suceder en otras situaciones también.

Recomendamos colocar puntos y coma entre las sentencias, incluso si están separadas por saltos de línea. Esta regla está ampliamente adoptada por la comunidad. Notemos una vez más que es posible omitir los puntos y coma la mayoría del tiempo. Pero es más seguro, especialmente para un principiante, usarlos.

Comentarios

A medida que pasa el tiempo, los programas se vuelven cada vez más complejos. Se hace necesario agregar *comentarios* que describan lo que hace el código y por qué.

Los comentarios se pueden poner en cualquier lugar de un script. No afectan su ejecución porque el motor simplemente los ignora.

Los comentarios de una línea comienzan con dos caracteres de barra diagonal `//`.

El resto de la línea es un comentario. Puede ocupar una línea completa propia o seguir una sentencia.

Como aquí:

```
1 // Este comentario ocupa una línea propia.
2 alert('Hello');
3
4 alert('World'); // Este comentario sigue a la sentenci
```

Los comentarios de varias líneas comienzan con una barra inclinada y un asterisco `/*` y terminan con un asterisco y una barra inclinada `*/`.

Como aquí:

```
1 /* Un ejemplo con dos mensajes.
2 Este es un comentario multilínea.
3 */
4 alert('Hola');
5 alert('Mundo');
```

El contenido de los comentarios se ignora, por lo que si colocamos el código dentro de `/* … */`, no se ejecutará.

A veces puede ser útil deshabilitar temporalmente una parte del código:

```
1 /* Comentando el código
2 alert('Hola');
3 */
4 alert('Mundo');
```

¡Usa accesos rápidos del teclado!

En la mayoría de los editores, se puede comentar una línea de código presionando `Ctrl+/` para un comentario de una sola línea y algo como `Ctrl+Shift+/` – para comentarios de varias líneas (selecciona una parte del código y pulsa la tecla de acceso rápido). Para Mac, intenta `Cmd` en lugar de `Ctrl` y `Option` en lugar de `Shift`.

¡Los comentarios anidados no son admitidos!

No puede haber `/*...*/` dentro de otro `/*...*/`.

Dicho código terminará con un error:

```
/*
  /* comentario anidado ?!? */
*/
alert( 'Mundo' );
```

Por favor, no dudes en comentar tu código.

Los comentarios aumentan el tamaño general del código, pero eso no es un problema en absoluto. Hay muchas herramientas que minimizan el código antes de publicarlo en un servidor de producción. Eliminan los comentarios, por lo que no aparecen en los scripts de trabajo. Por lo tanto, los comentarios no tienen ningún efecto negativo en la producción.

Más adelante, en el tutorial, habrá un capítulo Estilo de codificación que también explica cómo escribir mejores comentarios.

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

16 de junio de 2025

El modo moderno, "use strict"

Durante mucho tiempo, JavaScript evolucionó sin problemas de compatibilidad. Se añadían nuevas características al lenguaje sin que la funcionalidad existente cambiase.

Esto tenía el beneficio de nunca romper código existente, pero lo malo era que cualquier error o decisión incorrecta tomada por los creadores de JavaScript se quedaba para siempre en el lenguaje.

Esto fue así hasta 2009, cuando ECMAScript 5 (ES5) apareció. Esta versión añadió nuevas características al lenguaje y modificó algunas de las ya existentes. Para mantener el código antiguo funcionando, la mayoría de estas nuevas características están desactivadas por defecto. Tienes que activarlas explícitamente usando una directiva especial: `"use strict"`.

“use strict”

La directiva se presenta como un string: `"use strict"` o `'use strict'`. Cuando se sitúa al principio de un script, el script entero funciona de la manera “moderna”.

Por ejemplo:

```
"use strict";

// este código funciona de la manera moderna
...
```

Aprenderemos funciones (una manera de agrupar comandos) en breve, pero adelantemos que `"use strict"` se puede poner al inicio de una función. De esta manera, se activa el modo estricto únicamente en esa función. Pero normalmente se utiliza para el script entero.

Asegúrate de que “use strict” está al inicio

Por favor, asegúrate de que `"use strict"` está al principio de tus scripts. Si no, el modo estricto podría no estar activado.

El modo estricto no está activado aquí:

```
alert("algo de código");
// la directiva "use strict" de abajo es ignorad

"use strict";

// el modo estricto no está activado
```

Únicamente pueden aparecer comentarios por encima de `"use strict"`.

> **No hay manera de cancelar `use strict`**
>
> No hay ninguna directiva del tipo `"no use strict"` que haga al motor volver al comportamiento anterior.
>
> Una vez entramos en modo estricto, no hay vuelta atrás.

Consola del navegador

Cuando utilices la consola del navegador para ejecutar código, ten en cuenta que no utiliza `use strict` por defecto.

En ocasiones, donde `use strict` cause diferencia, obtendrás resultados incorrectos.

Entonces, ¿como utilizar `use strict` en la consola?

Primero puedes intentar pulsando `Shift+Enter` para ingresar múltiples líneas y poner `use strict` al principio, como aquí:

```
'use strict'; <Shift+Enter para una nueva línea>
//  ...tu código
<Intro para ejecutar>
```

Esto funciona para la mayoría de los navegadores, específicamente Firefox y Chrome.

Si esto no funciona, como en los viejos navegadores, hay una fea pero confiable manera de asegurar `use strict`. Ponlo dentro de esta especie de envoltura:

```
(function() {
  'use strict';

  // ...tu código...
})()
```

¿Deberíamos utilizar “use strict”?

La pregunta podría parecer obvia, pero no lo es.

Uno podría recomendar que se comiencen los script con `"use strict"` ... ¿Pero sabes lo que es interesante?

El JavaScript moderno admite “clases” y “módulos”, estructuras de lenguaje avanzadas (que seguramente llegaremos a ver), que automáticamente habilitan `use strict`. Entonces no necesitamos agregar la directiva `"use strict"` si las usamos.

Entonces, por ahora `"use strict";` es un invitado bienvenido al tope de tus scripts. Luego, cuando tu código sea todo clases y módulos, puedes omitirlo.

Hemos aprendido acerca de `use strict` en general.

En los siguientes capítulos, a medida que aprendamos características del lenguaje, veremos las diferencias entre el modo estricto y el antiguo. Afortunadamente no hay muchas y realmente hacen nuestra vida mejor.

Todos los ejemplos en este tutorial asumen modo estricto salvo que (muy raramente) se especifique lo contrario.

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

9 de marzo de 2026

Variables

La mayor parte del tiempo, una aplicación JavaScript necesita trabajar con información. Dos ejemplos:

1. Una tienda en línea – La información puede incluir los bienes a la venta y un "carrito de compras".
2. Una aplicación de chat – La información puede incluir los usuarios, mensajes, y mucho más.

Utilizamos las variables para almacenar esta información.

Una variable

Una variable es un "almacén con un nombre" para guardar datos. Podemos usar variables para almacenar golosinas, visitantes, y otros datos.

Para generar una variable en JavaScript, se usa la palabra clave `let`.

La siguiente declaración genera (en otras palabras: *declara* o *define*) una variable con el nombre "message":

```
let message;
```

Ahora podemos introducir datos en ella, utilizando el operador de asignación `=`:

```
let message;

message = 'Hola'; // almacenar la cadena 'Hola' en la
```

La cadena ahora está almacenada en el área de la memoria asociada con la variable. La podemos acceder utilizando el nombre de la variable:

```
let message;
message = 'Hola!';

alert(message); // muestra el contenido de la variable
```

Para ser concisos, podemos combinar la declaración de la variable y su asignación en una sola línea:

```
let message = 'Hola!'; // define la variable y asigna

```

```
alert(message); // Hola!
```

También podemos declarar múltiples variables en una sola línea:

```
let user = 'John', age = 25, message = 'Hola';
```

Esto puede parecer más corto, pero no lo recomendamos. Por el bien de la legibilidad, por favor utiliza una línea por variable.

La versión de líneas múltiples es un poco más larga, pero se lee más fácil:

```
let user = 'John';
let age = 25;
let message = 'Hola';
```

Hay quienes prefieren definir múltiples variables en estilo multilínea:

```
let user = 'John',
  age = 25,
  message = 'Hola';
```

... o con el estilo "coma primero":

```
let user = 'John'
  , age = 25
  , message = 'Hola';
```

Técnicamente, todas estas variantes hacen lo mismo. Así que, es cuestión de gusto personal y preferencia estética.

`var` en vez de `let`

En scripts más viejos, a veces se encuentra otra palabra clave: `var` en lugar de `let`:

```
var mensaje = 'Hola';
```

La palabra clave `var` es *casi* lo mismo que `let`. También hace la declaración de una variable, aunque de un modo ligeramente distinto, y más antiguo.

Existen sutiles diferencias entre `let` y `var`, pero no nos interesan en este momento. Cubriremos el tema a detalle en el capítulo La vieja "var".

Una analogía de la vida real

Podemos comprender fácilmente el concepto de una "variable" si nos la imaginamos como una "caja" con una etiqueta de nombre único pegada en ella.

Por ejemplo, la variable `message` puede ser imaginada como una caja etiquetada `"message"` con el valor `"Hola!"` dentro:

Podemos introducir cualquier valor a la caja.

También la podemos cambiar cuantas veces queramos:

```
let message;

message = 'Hola!';

message = 'Mundo!'; // valor alterado

alert(message);
```

Cuando el valor ha sido alterado, los datos antiguos serán removidos de la variable:

También podemos declarar dos variables y copiar datos de una a la otra.

```
let hello = 'Hola mundo!';

let message;

// copia 'Hola mundo' de hello a message
message = hello;

// Ahora, ambas variables contienen los mismos datos
alert(hello); // Hola mundo!
alert(message); // Hola mundo!
```

Declarar dos veces lanza un error

Una variable debe ser declarada solamente una vez.

Una declaración repetida de la misma variable es un error:

```
let message = "This";

// 'let' repetidos lleva a un error
let message = "That"; // SyntaxError: 'message'
```

Debemos declarar una variable una sola vez y desde entonces referirnos a ella sin `let`.

Lenguajes funcionales

Es interesante notar la existencia de la programación funcional. Los lenguajes funcionales "puros", como Haskell, prohíben cambiar el valor de las variables.

En tales lenguajes, una vez que la variable ha sido almacenada "en la caja", permanece allí por siempre. Si necesitamos almacenar algo más, el lenguaje nos obliga a crear una nueva caja (generar una nueva variable). No podemos reusar la antigua.

Aunque a primera vista puede parecer un poco extraño, estos lenguajes son muy capaces de desarrollo serio. Más aún: existen áreas, como la computación en paralelo, en las cuales esta limitación otorga ciertas ventajas.

Nombramiento de variables

Existen dos limitaciones de nombre de variables en JavaScript:

1. El nombre únicamente puede incluir letras, dígitos, o los símbolos `$` y `_`.
2. El primer carácter no puede ser un dígito.

Ejemplos de nombres válidos:

```
let userName;
let test123;
```

Cuando el nombre contiene varias palabras, se suele usar el estilo camelCase, donde las palabras van una detrás de otra y cada palabra, excepto la primera, empieza con mayúscula: `miNombreMuyLargo`.

Es interesante notar que el símbolo del dólar `'$'` y el guion bajo `'_'` también se utilizan en nombres. Son símbolos comunes, tal como las letras, sin ningún significado especial.

Los siguientes nombres son válidos:

```
let $ = 1; // Declara una variable con el nombre "$"
let _ = 2; // y ahora una variable con el nombre "_"

alert($ + _); // 3
```

Ejemplos de nombres incorrectos:

```
let 1a; // no puede iniciar con un dígito

let my-name; // los guiones '-' no son permitidos en n
```

La Capitalización es Importante

Dos variables con nombres `manzana` y `MANZANA` son variables distintas.

Las letras que no son del alfabeto inglés están permitidas, pero no se recomiendan

Es posible utilizar letras de cualquier alfabeto, incluyendo letras del cirílico, logogramas chinos, etc.:

```
let имя = '...';
let 我 = '...';
```

Técnicamente, no existe error aquí. Tales nombres están permitidos, pero internacionalmente existe la convención de utilizar el inglés para el nombre de las variables. Incluso si estamos escribiendo un script pequeño, este puede tener una larga vida por delante. Gente de otros países puede necesitar leerlo en algún momento.

Nombres reservados

Hay una lista de palabras reservadas, las cuales no pueden ser utilizadas como nombre de variable porque el lenguaje en sí las utiliza.

Por ejemplo: `let`, `class`, `return`, y `function` están reservadas.

El siguiente código nos da un error de sintaxis:

```
let let = 5; // no se puede le nombrar "let" a
let return = 5; // tampoco se le puede nombrar '
```

Una asignación sin utilizar `use strict`

Normalmente, debemos definir una variable antes de utilizarla. Pero, en los viejos tiempos, era técnicamente posible crear una variable simplemente asignando un valor sin utilizar 'let'. Esto aún funciona si no ponemos 'use strict' en nuestros scripts para mantener la compatibilidad con scripts antiguos.

```
// nota: no se utiliza "use strict" en este ejer

num = 5; // se crea la variable "num" si no exis

alert(num); // 5
```

Esto es una mala práctica que causaría errores en 'strict mode':

```
"use strict";

num = 5; // error: num no está definida
```

Constantes

Para declarar una variable constante (inmutable) use `const` en vez de `let`:

```
const myBirthday = '18.04.1982';
```

Las variables declaradas utilizando `const` se llaman "constantes". No pueden ser alteradas. Al intentarlo causaría un error:

```
const myBirthday = '18.04.1982';

myBirthday = '01.01.2001'; // ¡error, no se puede reas
```

Cuando un programador está seguro de que una variable nunca cambiará, puede declararla con `const` para garantizar esto y comunicarlo claramente a los demás.

Constantes mayúsculas

Existe una práctica utilizada ampliamente de utilizar constantes como aliases de valores difíciles-de-recordar y que se conocen previo a la ejecución.

Tales constantes se nombran utilizando letras mayúsculas y guiones bajos.

Por ejemplo, creemos constantes para los colores en el formato "web" (hexadecimal):

```
const COLOR_RED = "#F00";
const COLOR_GREEN = "#0F0";
const COLOR_BLUE = "#00F";
const COLOR_ORANGE = "#FF7F00";

// ...cuando debemos elegir un color
let color = COLOR_ORANGE;
alert(color); // #FF7F00
```

Ventajas:

- `COLOR_ORANGE` es mucho más fácil de recordar que `"#FF7F00"`.
- Es mucho más fácil escribir mal `"#FF7F00"` que `COLOR_ORANGE`.
- Al leer el código, `COLOR_ORANGE` tiene mucho más significado que `#FF7F00`.

¿Cuándo se deben utilizar letras mayúsculas para una constante, y cuando se debe nombrarla de manera normal? Dejémoslo claro.

Ser una "constante" solo significa que el valor de la variable nunca cambia. Pero hay constantes que son conocidas previo a la ejecución (como el valor hexadecimal del color rojo) y hay constantes que son *calculadas* en el tiempo de ejecución, pero no cambian después de su asignación inicial.

Por ejemplo:

```
1 const pageLoadTime = /* el tiempo que tardó la página
```

El valor de `pageLoadTime` no está preestablecido. Como no se conoce antes de cargar la página, la nombramos normalmente. Pero podemos declararla como constante, porque después de su asignación inicial, no cambiará.

En otras palabras, las constantes en mayúsculas son utilizadas solamente como alias para valores invariables y preestablecidos.

Nombrar cosas correctamente

Estando en el tema de las variables, existe una cosa de mucha importancia.

Una variable debe tener un nombre claro, de significado evidente, que describa el dato que almacena.

Nombrar variables es una de las habilidades más importantes y complejas en la programación. Un vistazo rápido a el nombre de las variables nos revela cuál código fue escrito por un principiante o por un desarrollador experimentado.

En un proyecto real, se pasa mucho más tiempo modificando y extendiendo una base de código existente que escribiendo algo nuevo desde cero. Cuando regresamos a nuestro código luego de un tiempo, es mucho más fácil encontrar información que está bien etiquetada. O en otras palabras, cuando las variables tienen los nombres adecuados.

Por favor, dedica tiempo para pensar un nombre correcto para una variable antes de declararla. Hacer esto te rendirá muy bien.

Algunas reglas buenas para seguir:

- Usa términos legibles para humanos como `userName` p `shoppingCart`.
- Evita abreviaciones o nombres cortos `a`, `b`, `c`, a menos que realmente sepas lo que estás haciendo.
- Crea nombres que describan al máximo lo que son y sean concisos. Ejemplos de nombres malos son `data` y `value`. Estos nombres no nos dicen nada, solo son adecuados en el contexto de un código que deje excepcionalmente obvio cuál dato o valor está referenciando la variable.
- Ponte de acuerdo con tu equipo, y con tu propia mente, cuáles términos se utilizarán. Si a un visitante se lo llamara "user", debemos llamar las variables relacionadas `currentUser` o `newUser` en vez de `currentVisitor` o `newManInTown`.

¿Suena simple? De hecho lo es, pero no es tan fácil crear nombres de variables descriptivos y concisos a la hora de practicar. Inténtelo.

> **¿Reusar o crear?**
>
> Una última nota. Existen programadores haraganes que, en vez de declarar una variable nueva, tienden a reusar las existentes.
>
> El resultado de esto es que sus variables son como cajas en las cuales la gente introduce cosas distintas sin cambiar sus etiquetas. ¿Que existe dentro de la caja? ¿Quién sabe? Necesitamos acercarnos y revisar.
>
> Dichos programadores se ahorran un poco durante la declaración de la variable, pero pierden diez veces más a la hora de depuración.
>
> Una variable extra es algo bueno, no algo diabólico.
>
> Los navegadores modernos y los minificadores de JavaScript optimizan el código, así que esto no impacta en el rendimiento. Utilizar diferentes variables para distintos valores incluso puede ayudar a optimizar tu código.

Resumen

Podemos declarar variables para almacenar datos al utilizar las palabra clave `var`, `let`, o `const`.

- `let` – es la forma moderna de declaración de una variable.
- `var` – es la declaración de variable de vieja escuela. Normalmente no lo utilizamos en absoluto. Cubriremos sus sutiles diferencias con `let` en el capítulo La vieja "var", por si lo necesitaras.
- `const` – es como `let`, pero una vez asignado, el valor de la variable no podrá alterarse.

Las variables deben ser nombradas de tal manera que entendamos fácilmente lo que está en su interior.

Tareas

Trabajando con variables.

importancia: 2

1. Declara dos variables: `admin` y `name`.
2. Asigna el valor `"John"` a `name`.
3. Copia el valor de `name` a `admin`.
4. Muestra el valor de `admin` usando `alert` (debe salir “John”).

solución

Dando el nombre correcto

importancia: 3

1. Crea una variable con el nombre de nuestro planeta. ¿Cómo nombrarías a dicha variable?

2. Crea una variable para almacenar el nombre del usuario actual de un sitio web. ¿Cómo nombrarías a dicha variable?

solución

¿const mayúsculas?

importancia: 4

Examina el siguiente código:

```
const birthday = '18.04.1982';

const age = someCode(birthday);
```

Aquí tenemos una constante `birthday` para la fecha de cumpleaños, y la edad `age`, que también es constante.

`age` es calculada desde `birthday` con la ayuda de "cierto código" `someCode()`, que es una llamada a función que no hemos explicado aún (¡lo haremos pronto!); los detalles no importan aquí, el punto es que `age` se calcula de alguna forma basándose en `birthday`.

¿Sería correcto usar mayúsculas en `birthday`? ¿Y en `age`? ¿O en ambos?

```
const BIRTHDAY = '18.04.1982'; // ¿birthday en mayúscu

const AGE = someCode(BIRTHDAY); // ¿age en mayúsculas?
```

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor acerca del proyecto contáctenos

22 de agosto de 2024

Tipos de datos

Un valor en JavaScript siempre pertenece a un tipo de dato determinado. Por ejemplo, un string o un número.

Hay ocho tipos de datos básicos en JavaScript. En este capítulo los cubriremos en general y en los próximos hablaremos de cada uno de ellos en detalle.

Podemos almacenar un valor de cualquier tipo dentro de una variable. Por ejemplo, una variable puede contener en un momento un string y luego almacenar un número:

```
// no hay error
let message = "hola";
message = 123456;
```

Los lenguajes de programación que permiten estas cosas, como JavaScript, se denominan "dinámicamente tipados", lo que significa que allí hay tipos de datos, pero las variables no están vinculadas rígidamente a ninguno de ellos.

Number

```
let n = 123;
n = 12.345;
```

El tipo *number* representa tanto números enteros como de punto flotante.

Hay muchas operaciones para números. Por ejemplo, multiplicación `*`, división `/`, suma `+`, resta `-`, y demás.

Además de los números comunes, existen los llamados "valores numéricos especiales" que también pertenecen a este tipo de datos: `Infinity`, `-Infinity` y `NaN`.

- `Infinity` representa el Infinito matemático ∞. Es un valor especial que es mayor que cualquier número.

  Podemos obtenerlo como resultado de la división por cero:

  ```
  alert( 1 / 0 ); // Infinity
  ```

  O simplemente hacer referencia a él directamente:

  ```
  alert( Infinity ); // Infinity
  ```

- `NaN` representa un error de cálculo. Es el resultado de una operación matemática incorrecta o indefinida, por ejemplo:

```
alert( "no es un número" / 2 ); // NaN, tal división
```

`NaN` es "pegajoso". Cualquier otra operación sobre `NaN` devuelve `NaN`:

```
alert( NaN + 1 ); // NaN
alert( 3 * NaN ); // NaN
alert( "not a number" / 2 - 1 ); // NaN
```

Por lo tanto, si hay un `NaN` en alguna parte de una expresión matemática, se propaga a todo el resultado (con una única excepción: `NaN ** 0` es `1`).

> **Las operaciones matemáticas son seguras**
>
> Hacer matemáticas es "seguro" en JavaScript. Podemos hacer cualquier cosa: dividir por cero, tratar las cadenas no numéricas como números, etc.
>
> El script nunca se detendrá con un error fatal (y "morir"). En el peor de los casos, obtendremos `NaN` como resultado.

Los valores numéricos especiales pertenecen formalmente al tipo "número". Por supuesto que no son números en el sentido estricto de la palabra.

Veremos más sobre el trabajo con números en el capítulo Números.

BigInt

En JavaScript, el tipo "number" no puede representar de forma segura valores enteros mayores que $(2^{53}-1)$ (eso es `9007199254740991`), o menor que $-(2^{53}-1)$ para negativos.

Para ser realmente precisos, el tipo de dato "number" puede almacenar enteros muy grandes (hasta $1.7976931348623157 * 10^{308}$), pero fuera del rango de enteros seguros $\pm(2^{53}-1)$ habrá un error de precisión, porque no todos los dígitos caben en el almacén fijo de 64-bit. Así que es posible que se almacene un valor "aproximado".

Por ejemplo, estos dos números (justo por encima del rango seguro) son iguales:

```
console.log(9007199254740991 + 1); // 9007199254740992
console.log(9007199254740991 + 2); // 9007199254740992
```

Podemos decir que ningún entero impar mayor que $(2^{53}-1)$ puede almacenarse en el tipo de dato "number".

Para la mayoría de los propósitos, el rango $\pm(2^{53}-1)$ es suficiente, pero a veces necesitamos números realmente grandes; por ejemplo, para criptografía o marcas de tiempo de precisión de microsegundos.

`BigInt` se agregó recientemente al lenguaje para representar enteros de longitud arbitraria.

Un valor `BigInt` se crea agregando `n` al final de un entero:

```
// la "n" al final significa que es un BigInt
const bigInt = 1234567890123456789012345678901234567890
```

Como los números `BigInt` rara vez se necesitan, no los cubrimos aquí sino que les dedicamos un capítulo separado <info: bigint>. Léelo cuando necesites números tan grandes.

String

Un *string* en JavaScript es una cadena de caracteres y debe colocarse entre comillas.

```
let str = "Hola";
let str2 = 'Las comillas simples también están bien';
let phrase = `se puede incrustar otro ${str}`;
```

En JavaScript, hay 3 tipos de comillas.

1. Comillas dobles: `"Hola"`.
2. Comillas simples: `'Hola'`.
3. Backticks (comillas invertidas): `` `Hola` ``.

Las comillas dobles y simples son comillas "sencillas" (es decir, funcionan igual). No hay diferencia entre ellas en JavaScript.

Los backticks son comillas de "funcionalidad extendida". Nos permiten incrustar variables y expresiones en una cadena de caracteres encerrándolas en `${...}`, por ejemplo:

```
let name = "John";

// incrustar una variable
alert( `Hola, ${name}!` ); // Hola, John!

// incrustar una expresión
alert( `el resultado es ${1 + 2}` ); //el resultado es
```

La expresión dentro de `${...}` se evalúa y el resultado pasa a formar parte de la cadena. Podemos poner cualquier cosa ahí dentro: una variable como `name`, una expresión aritmética como `1 + 2`, o algo más complejo.

Toma en cuenta que esto sólo se puede hacer con los backticks. ¡Las otras comillas no tienen esta capacidad de incrustación!

```
alert( "el resultado es ${1 + 2}" ); // el resultado e
```

En el capítulo Strings trataremos más a fondo las cadenas.

> **No existe el tipo *carácter***
>
> En algunos lenguajes, hay un tipo especial "carácter" para un solo carácter. Por ejemplo, en el lenguaje C y en Java es `char`.
>
> En JavaScript no existe tal tipo. Sólo hay un tipo: `string`. Un string puede estar formado por un solo carácter, por ninguno, o por varios de ellos.

Boolean (tipo lógico)

El tipo *boolean* tiene sólo dos valores posibles: `true` y `false`.

Este tipo se utiliza comúnmente para almacenar valores de sí/no: `true` significa "sí, correcto, verdadero", y `false` significa "no, incorrecto, falso".

Por ejemplo:

```
let nameFieldChecked = true; // sí, el campo name está
let ageFieldChecked = false; // no, el campo age no es
```

Los valores booleanos también son el resultado de comparaciones:

```
let isGreater = 4 > 1;

alert( isGreater ); // verdadero (el resultado de la c
```

En el capítulo Operadores Lógicos trataremos más a fondo el tema de los booleanos.

El valor "null" (nulo)

El valor especial `null` no pertenece a ninguno de los tipos descritos anteriormente.

Forma un tipo propio separado que contiene sólo el valor `null`:

```
let age = null;
```

En JavaScript, `null` no es una "referencia a un objeto inexistente" o un "puntero nulo" como en otros lenguajes.

Es sólo un valor especial que representa "nada", "vacío" o "valor desconocido".

El código anterior indica que el valor de `age` es desconocido o está vacío por alguna razón.

El valor “undefined” (indefinido)

El valor especial `undefined` también se distingue. Hace un tipo propio, igual que `null`.

El significado de `undefined` es “valor no asignado”.

Si una variable es declarada, pero no asignada, entonces su valor es `undefined`:

```
let age;

alert(age); // muestra "undefined"
```

Técnicamente, es posible asignar `undefined` a cualquier variable:

```
let age = 100;

// cambiando el valor a undefined
age = undefined;

alert(age); // "undefined"
```

...Pero no recomendamos hacer eso. Normalmente, usamos `null` para asignar un valor “vacío” o “desconocido” a una variable, mientras `undefined` es un valor inicial reservado para cosas que no han sido asignadas.

Object y Symbol

El tipo `object` (objeto) es especial.

Todos los demás tipos se llaman “primitivos” porque sus valores pueden contener una sola cosa (ya sea una cadena, un número, o lo que sea). Por el contrario, los objetos se utilizan para almacenar colecciones de datos y entidades más complejas.

Siendo así de importantes, los objetos merecen un trato especial. Nos ocuparemos de ellos más adelante en el capítulo Objetos después de aprender más sobre los primitivos.

El tipo `symbol` (símbolo) se utiliza para crear identificadores únicos para los objetos. Tenemos que mencionarlo aquí para una mayor integridad, pero es mejor estudiar este tipo después de los objetos.

El operador typeof

El operador `typeof` devuelve el tipo de dato del operando. Es útil cuando queremos procesar valores de diferentes tipos de forma diferente o simplemente queremos hacer una comprobación rápida.

La llamada a `typeof x` devuelve una cadena con el nombre del tipo:

```
typeof undefined // "undefined"

```

```
typeof 0 // "number"

typeof 10n // "bigint"

typeof true // "boolean"

typeof "foo" // "string"

typeof Symbol("id") // "symbol"

typeof Math // "object"  (1)

typeof null // "object"  (2)

typeof alert // "function"  (3)
```

Las últimas tres líneas pueden necesitar una explicación adicional:

1. `Math` es un objeto incorporado que proporciona operaciones matemáticas. Lo aprenderemos en el capítulo Números. Aquí sólo sirve como ejemplo de un objeto.
2. El resultado de `typeof null` es `"object"`. Esto está oficialmente reconocido como un error de comportamiento de `typeof` que proviene de los primeros días de JavaScript y se mantiene por compatibilidad. Definitivamente `null` no es un objeto. Es un valor especial con un tipo propio separado.
3. El resultado de `typeof alert` es `"function"` porque `alert` es una función. Estudiaremos las funciones en los próximos capítulos donde veremos que no hay ningún tipo especial "function" en JavaScript. Las funciones pertenecen al tipo objeto. Pero `typeof` las trata de manera diferente, devolviendo `function`. Además proviene de los primeros días de JavaScript. Técnicamente dicho comportamiento es incorrecto, pero puede ser conveniente en la práctica.

Sintaxis de `typeof(x)`

Se puede encontrar otra sintaxis en algún código: `typeof(x)`. Es lo mismo que `typeof x`.

Para ponerlo en claro: `typeof` es un operador, no una función. Los paréntesis aquí no son parte del operador `typeof`. Son del tipo usado en agrupamiento matemático.

Usualmente, tales paréntesis contienen expresiones matemáticas tales como `(2 + 2)`, pero aquí solo tienen un argumento `(x)`. Sintácticamente, permiten evitar el espacio entre el operador `typeof` y su argumento, y a algunas personas les gusta así.

Algunos prefieren `typeof(x)`, aunque la sintaxis `typeof x` es mucho más común.

Resumen

Hay 8 tipos básicos en JavaScript.

- Siete tipos de datos primitivos

- `number` para números de cualquier tipo: enteros o de punto flotante, los enteros están limitados por $\pm(2^{53}-1)$.
- `bigint` para números enteros de longitud arbitraria.
- `string` para cadenas. Una cadena puede tener cero o más caracteres, no hay un tipo especial para un único carácter.
- `boolean` para verdadero y falso: `true` / `false`.
- `null` para valores desconocidos – un tipo independiente que tiene un solo valor nulo: `null`.
- `undefined` para valores no asignados – un tipo independiente que tiene un único valor "indefinido": `undefined`.
- `symbol` para identificadores únicos.

- Y un tipo de dato no primitivo:
  - `object` para estructuras de datos complejas.

El operador `typeof` nos permite ver qué tipo está almacenado en una variable.

- Dos formas: `typeof x` o `typeof(x)`.
- Devuelve una cadena con el nombre del tipo. Por ejemplo `"string"`.
- Para `null` devuelve `"object"`: esto es un error en el lenguaje, en realidad no es un objeto.

Tareas

Comillas

importancia: 5

¿Cuál es la salida del script?

```
let name = "Ilya";

alert( `Hola ${1}` ); // ?

alert( `Hola ${"name"}` ); // ?

alert( `Hola ${name}` ); // ?
```

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor acerca del proyecto contáctenos

contáctenos

5 de febrero de 2023

Interacción: alert, prompt, confirm

Como usaremos el navegador como nuestro entorno de demostración, veamos un par de funciones para interactuar con el usuario: `alert`, `prompt`, y `confirm`.

alert

Ya la hemos visto. Muestra un mensaje y espera a que el usuario presione "Aceptar".

Por ejemplo:

```
1 alert("Hello");
```

La mini ventana con el mensaje se llama * ventana modal *. La palabra "modal" significa que el visitante no puede interactuar con el resto de la página, presionar otros botones, etc., hasta que se haya ocupado de la ventana. En este caso, hasta que presionen "OK".

prompt

La función `prompt` acepta dos argumentos:

```
1 result = prompt(title, [default]);
```

Muestra una ventana modal con un mensaje de texto, un campo de entrada para el visitante y los botones OK/CANCELAR.

`title`

El texto a mostrar al usuario.

`default`

Un segundo parámetro opcional, es el valor inicial del campo de entrada.

> **Corchetes en la sintaxis `[...]`**
>
> Los corchetes alrededor de `default` en la sintaxis de arriba denotan que el parámetro es opcional, no requerido.

El usuario puede escribir algo en el campo de entrada de solicitud y presionar OK, así obtenemos ese texto en `result`. O puede cancelar la entrada, con el

botón “Cancelar” o presionando la tecla `Esc`, de este modo se obtiene `null` en `result`.

La llamada a `prompt` retorna el texto del campo de entrada o `null` si la entrada fue cancelada.

Por ejemplo:

```
let age = prompt ('¿Cuántos años tienes?', 100);

alert(`Tienes ${age} años!`); //Tienes 100 años!
```

En IE: proporciónale un *predeterminado* siempre

El segundo parámetro es opcional, pero si no lo proporcionamos, Internet Explorer insertará el texto `"undefined"` en el prompt.

Ejecuta este código en Internet Explorer para verlo:

```
let test = prompt("Test");
```

Por lo tanto, para que las indicaciones se vean bien en IE, recomendamos siempre proporcionar el segundo argumento:

```
let test = prompt("Test", ''); // <-- para IE
```

confirm

La sintaxis:

```
result = confirm(pregunta);
```

La función `confirm` muestra una ventana modal con una `pregunta` y dos botones: OK y CANCELAR.

El resultado es `true` si se pulsa OK y `false` en caso contrario.

Por ejemplo:

```
let isBoss = confirm("¿Eres el jefe?");

alert( isBoss ); // true si se pulsa OK
```

Resumen

Cubrimos 3 funciones específicas del navegador para interactuar con los usuarios:

`alert`

muestra un mensaje.

`prompt`

muestra un mensaje pidiendo al usuario que introduzca un texto. Retorna el texto o, si se hace clic en CANCELAR o se presiona `Esc`, retorna `null`.

`confirm`

muestra un mensaje y espera a que el usuario pulse “OK” o “CANCELAR”. Retorna `true` si se presiona OK y `false` si se presiona CANCEL/`Esc`.

Todos estos métodos son modales: detienen la ejecución del script y no permiten que el usuario interactúe con el resto de la página hasta que la ventana se haya cerrado.

Hay dos limitaciones comunes a todos los métodos anteriores:

1. La ubicación exacta de la ventana modal está determinada por el navegador. Normalmente, está en el centro.
2. El aspecto exacto de la ventana también depende del navegador. No podemos modificarlo.

Ese es el precio de la simplicidad. Existen otras formas de mostrar ventanas más atractivas e interactivas para el usuario, pero si la apariencia no importa mucho, estos métodos funcionan bien.

Tareas

Una página simple

importancia: 4

Crea una página web que pida un nombre y lo muestre.

Ejecutar el demo

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor acerca del proyecto contáctenos

22 de septiembre de 2025

Conversiones de Tipos

La mayoría de las veces, los operadores y las funciones convierten automáticamente los valores que reciben al tipo de dato que necesitan.

Por ejemplo: `alert` convierte automáticamente cualquier valor a string para mostrarlo, y las operaciones matemáticas convierten los valores a números.

También hay casos donde necesitamos convertir de manera explícita un valor al tipo esperado.

> **No hablaremos de los objetos aún**
>
> En este capítulo no cubrimos objetos. Por ahora, solamente veremos valores primitivos.
>
> Más adelante, después de haberlos tratado, veremos en el capítulo Conversión de objeto a valor primitivo cómo funciona la conversión.

Conversión a String

La conversión a string ocurre cuando necesitamos la representación textual de un valor.

Por ejemplo, `alert(value)` lo hace para mostrar `value` como texto.

También podemos llamar a la función `String(value)` para convertir `value` a string:

```
1 let value = true;
2 alert(typeof value); // boolean
3
4 value = String(value); // ahora value es el string "tr
5 alert(typeof value); // string
```

La conversión a string es bastante evidente. El boolean `false` se convierte en `"false"`, `null` en `"null"`, etc.

Conversión numérica

La conversión numérica ocurre automáticamente en funciones matemáticas y expresiones.

Por ejemplo, cuando se aplica la división `/` a valores no numéricos:

```
alert( "6" / "2" ); // 3, los strings son convertidos
```

Podemos usar la función `Number(value)` para convertir `value` a un número de forma explícita:

```
let str = "123";
alert(typeof str); // string

let num = Number(str); // se convierte en 123

alert(typeof num); // number
```

Usualmente, se requiere conversión explícita cuando leemos un valor desde una fuente basada en texto, como un formulario, pero esperamos que sea un número.

Si el string no es un número válido, el resultado de la conversión será `NaN`. Por ejemplo:

```
let age = Number("un texto arbitrario en vez de un núm

alert(age); // NaN, conversión fallida
```

Reglas de conversión numérica:

Valor	Se convierte en...
`undefined`	`NaN`
`null`	`0`
`true and false`	`1` y `0`
`string`	Se eliminan los espacios en blanco (incluyendo espacios, tabs `\t`, saltos de línea `\n`, etc.) del inicio y final del texto. Si el string resultante es vacío, el resultado es `0`, en caso contrario el número es "leído" del string. Un error devuelve `NaN`.

Ejemplos:

```
alert( Number("   123   ") ); // 123
alert( Number("123z") );      // NaN (error al leer el
alert( Number(true) );        // 1
alert( Number(false) );       // 0
```

Es importante notar que `null` y `undefined` se comportan de distinta manera aquí: `null` se convierte en `0` mientras que `undefined` se convierte en `NaN`.

La mayoría de los operadores matemáticos también ejecutan dicha conversión, esto lo veremos en el siguiente capítulo.

Conversión booleana

La conversión a boolean es la más simple.

Ocurre en operaciones lógicas (más adelante veremos test condicionales y otras cosas similares), pero también puede realizarse de forma explícita llamando a la función `Boolean(value)`.

Las reglas de conversión:

- Los valores que son intuitivamente "vacíos", como `0`, `""`, `null`, `undefined`, y `NaN`, se convierten en `false`.
- Otros valores se convierten en `true`.

Por ejemplo:

```
alert( Boolean(1) ); // true
alert( Boolean(0) ); // false

alert( Boolean("hola") ); // true
alert( Boolean("") ); // false
```

Ten en cuenta: el string con un cero `"0"` es `true`

Algunos lenguajes (como PHP) tratan `"0"` como `false`. Pero en JavaScript, un string no vacío es siempre `true`.

```
alert( Boolean("0") ); // true
alert( Boolean(" ") ); // sólo espacios, también
```

Resumen

Las tres conversiones de tipo más usadas son a string, a número y a boolean.

`Conversión a String` – Ocurre cuando mostramos un valor. Se puede realizar con `String(value)`. La conversión a string es usualmente evidente para los valores primitivos.

`Conversión numérica` – Ocurre en operaciones matemáticas. Se puede realizar con `Number(value)`.

La conversión sigue las reglas:

Valor	Se convierte en...
`undefined`	`NaN`
`null`	`0`
`true / false`	`1 / 0`
`string`	El string es leído "como es", los espacios en blanco (incluye espacios, tabs `\t`, saltos de línea `\n`, etc.) tanto al inicio como al final son ignorados. Un string vacío se convierte en `0`. Un error entrega `NaN`.

`Conversión booleana` – Ocurren en operaciones lógicas. Se puede realizar con `Boolean(value)`.

Sigue las reglas:

Valor	Se convierte en...
`0`, `null`, `undefined`, `NaN`, `""`	`false`
cualquier otro valor	`true`

La mayoría de estas reglas son fáciles de entender y recordar. Las excepciones más notables donde se suele cometer errores son:

- `undefined` es `NaN` como número, no `0`.
- `"0"` y textos que solo contienen espacios como `" "` son `true` como boolean.

Los objetos no son cubiertos aquí. Volveremos a ellos más tarde en el capítulo Conversión de objeto a valor primitivo que está dedicado exclusivamente a objetos después de que aprendamos más cosas básicas sobre JavaScript.

Comentarios

lea esto antes de comentar...



© 2007—2026 Ilya Kantor

9 de marzo de 2026

Operadores básicos, matemáticas

Conocemos varios operadores matemáticos porque nos los enseñaron en la escuela. Son cosas como la suma `+`, multiplicación `*`, resta `-`, etcétera.

En este capítulo, nos vamos a concentrar en los aspectos de los operadores que no están cubiertos en la aritmética escolar.

Términos: "unario", "binario", "operando"

Antes de continuar, comprendamos la terminología común.

- *Un operando* – es a lo que se aplican los operadores. Por ejemplo, en la multiplicación de `5 * 2` hay dos operandos: el operando izquierdo es `5` y el operando derecho es `2`. A veces, la gente los llama "argumentos" en lugar de "operandos".
- Un operador es *unario* si tiene un solo operando. Por ejemplo, la negación unaria `-` invierte el signo de un número:

```
let x = 1;

x = -x;
alert( x ); // -1, se aplicó negación unaria
```

- Un operador es *binario* si tiene dos operandos. El mismo negativo también existe en forma binaria:

```
let x = 1, y = 3;
alert( y - x ); // 2, binario negativo resta valores
```

Formalmente, estamos hablando de dos operadores distintos: la negación unaria (un operando: revierte el símbolo) y la resta binaria (dos operandos: resta).

Matemáticas

Están soportadas las siguientes operaciones:

- Suma `+`,
- Resta `-`,
- Multiplicación `*`,
- División `/`,
- Resto `%`,

- Exponenciación `**`.

Los primeros cuatro son conocidos mientras que `%` y `**` deben ser explicados más ampliamente.

Resto %

El operador resto `%`, a pesar de su apariencia, no está relacionado con porcentajes.

El resultado de `a % b` es el resto de la división entera de `a` por `b`.

Por ejemplo:

```
alert( 5 % 2 ); // 1, es el resto de 5 dividido por 2
alert( 8 % 3 ); // 2, es el resto de 8 dividido por 3
alert( 8 % 4 ); // 0, es el resto de 8 dividido por 4
```

Exponenciación **

El operador exponenciación `a ** b` eleva `a` a la potencia de `b`.

En matemáticas de la escuela, lo escribimos como a^b.

Por ejemplo:

```
alert( 2 ** 2 ); // 2² = 4
alert( 2 ** 3 ); // 2³ = 8
alert( 2 ** 4 ); // 2⁴ = 16
```

Matemáticamente, la exponenciación está definida para operadores no enteros también.

Por ejemplo, la raíz cuadrada es el exponente ½:

```
alert( 4 ** (1/2) ); // 2 (potencia de 1/2 es lo mismo
alert( 8 ** (1/3) ); // 2 (potencia de 1/3 es lo mismo
```

Concatenación de cadenas con el binario +

Ahora veamos las características de los operadores de JavaScript que van más allá de la aritmética escolar.

Normalmente el operador `+` suma números.

Pero si se aplica el `+` binario a una cadena, los une (concatena):

```
let s = "my" + "string";
alert(s); // mystring
```

Tenga presente que si uno de los operandos es una cadena, el otro es convertido a una cadena también.

Por ejemplo:

```
alert( '1' + 2 ); // "12"
alert( 2 + '1' ); // "21"
```

Vieron, no importa si el primer operando es una cadena o el segundo.

Aquí hay un ejemplo algo más complejo:

```
alert(2 + 2 + '1' ); // "41" y no "221"
```

Aquí, los operadores trabajan uno después de otro. El primer `+` suma dos números entonces devuelve `4`, luego el siguiente `+` le agrega la cadena `1`, así que se evalúa como `4 + '1' = 41`.

```
alert('1' + 2 + 2); // "122", no es "14"
```

Aquí el primer operando es una cadena, el compilador trata los otros dos operandos como cadenas también. El `2` es concatenado a `'1'`, entonces es como `'1' + 2 = "12"` y `"12" + 2 = "122"`.

El binario `+` es el único operador que soporta cadenas en esa forma. Otros operadores matemáticos trabajan solamente con números y siempre convierten sus operandos a números.

Por ejemplo, resta y división:

```
alert( 2 - '1' ); // 1
alert( '6' / '2' ); // 3
```

Conversión numérica, unario +

La suma `+` existe en dos formas: la forma binaria que utilizamos arriba y la forma unaria.

El unario suma o, en otras palabras, el operador suma `+` aplicado a un solo valor, no hace nada a los números. Pero si el operando no es un número, el unario suma lo convierte en un número.

Por ejemplo:

```
// Sin efecto en números
let x = 1;
alert( +x ); // 1

let y = -2;
alert( +y ); // -2

// Convierte los no números
alert( +true ); // 1
alert( +"" );   // 0
```

Realmente hace lo mismo que `Number(...)`, pero es más corto.

La necesidad de convertir cadenas en números surge con mucha frecuencia. Por ejemplo, si estamos obteniendo valores de campos de formulario HTML, generalmente son cadenas.

El operador binario suma los agregaría como cadenas:

```
let apples = "2";
let oranges = "3";

alert( apples + oranges ); // "23", el binario suma co
```

Si queremos tratarlos como números, necesitamos convertirlos y luego sumarlos:

```
let apples = "2";
let oranges = "3";

// ambos valores convertidos a números antes del opera
alert( +apples + +oranges ); // 5

// la variante más larga
// alert( Number(apples) + Number(oranges) ); // 5
```

Desde el punto de vista de un matemático, la abundancia de signos más puede parecer extraña. Pero desde el punto de vista de un programador no hay nada especial: primero se aplican los signos más unarios que convierten las cadenas en números, y luego el signo más binario los suma.

¿Por qué se aplican los signos más unarios a los valores antes que los binarios? Como veremos, eso se debe a su *mayor precedencia*.

Precedencia del operador

Si una expresión tiene más de un operador, el orden de ejecución se define por su *precedencia* o, en otras palabras, el orden de prioridad predeterminado de los operadores.

Desde la escuela, todos sabemos que la multiplicación en la expresión `1 + 2 * 2` debe calcularse antes de la suma. Eso es exactamente la precedencia. Se dice que la multiplicación tiene *una mayor precedencia* que la suma.

Los paréntesis anulan cualquier precedencia, por lo que si no estamos satisfechos con el orden predeterminado, podemos usarlos para cambiarlo. Por ejemplo, escriba `(1 + 2) * 2`.

Hay muchos operadores en JavaScript. Cada operador tiene un número de precedencia correspondiente. El que tiene el número más grande se ejecuta primero. Si la precedencia es la misma, el orden de ejecución es de izquierda a derecha.

Aquí hay un extracto de la tabla de precedencia (no necesita recordar esto, pero tenga en cuenta que los operadores unarios son más altos que el operador binario correspondiente):

Precedencia	Nombre	Signo

...	...	...
14	suma unaria	`+`
14	negación unaria	`-`
13	exponenciación	`**`
12	multiplicación	`*`
12	división	`/`
11	suma	`+`
11	resta	`-`
...	...	...
2	asignación	`=`
...	...	...

Como podemos ver, la "suma unaria" tiene una prioridad de `14`, que es mayor que el `11` de "suma" (suma binaria). Es por eso que, en la expresión `"+apples + +oranges"`, las sumas unarias se hacen antes de la adición.

Asignación

Tengamos en cuenta que una asignación `=` también es un operador. Está listado en la tabla de precedencia con la prioridad muy baja de `2`.

Es por eso que, cuando asignamos una variable, como `x = 2 * 2 + 1`, los cálculos se realizan primero y luego se evalúa el `=`, almacenando el resultado en `x`.

```
let x = 2 * 2 + 1;

alert( x ); // 5
```

Asignación = devuelve un valor

El hecho de que `=` sea un operador, no una construcción "mágica" del lenguaje, tiene un implicación interesante.

Todos los operadores en JavaScript devuelven un valor. Esto es obvio para `+` y `-`, pero también es cierto para `=`.

La llamada `x = value` escribe el `value` en `x` *y luego lo devuelve*.

Aquí hay una demostración que usa una asignación como parte de una expresión más compleja:

```
let a = 1;
let b = 2;

let c = 3 - (a = b + 1);

alert( a ); // 3
alert( c ); // 0
```

En el ejemplo anterior, el resultado de la expresión `(a = b + 1)` es el valor asignado a `a` (es decir, `3`). Luego se usa para evaluaciones adicionales.

Código gracioso, ¿no? Deberíamos entender cómo funciona, porque a veces lo vemos en las bibliotecas de JavaScript.

Pero no deberíamos escribir algo así. Tales trucos definitivamente no hacen que el código sea más claro o legible.

Asignaciones encadenadas

Otra característica interesante es la habilidad para encadenar asignaciones:

```
let a, b, c;

a = b = c = 2 + 2;

alert( a ); // 4
alert( b ); // 4
alert( c ); // 4
```

Las asignaciones encadenadas evalúan de derecha a izquierda. Primero, se evalúa la expresión más a la derecha `2 + 2` y luego se asigna a las variables de la izquierda: `c`, `b` y `a`. Al final, todas las variables comparten un solo valor.

Una vez más, por motivos de legibilidad, es mejor separar este código en varias líneas:

```
c = 2 + 2;
b = c;
a = c;
```

Es más fácil de leer, especialmente cuando se hace de un vistazo.

Modificar en el lugar

A menudo necesitamos aplicar un operador a una variable y guardar el nuevo resultado en esa misma variable.

Por ejemplo:

```
let n = 2;
n = n + 5;
n = n * 2;
```

Esta notación puede ser acortada utilizando los operadores `+=` y `*=`:

```
let n = 2;
n += 5; // ahora n = 7  (es lo mismo que n = n + 5)
n *= 2; // ahora n = 14 (es lo mismo que n = n * 2)

alert( n ); // 14
```

Los operadores cortos "modifica y asigna" existen para todos los operadores aritméticos y de nivel bit: `/=`, `-=`, etcétera.

Tales operadores tienen la misma precedencia que la asignación normal, por lo tanto se ejecutan después de otros cálculos:

```
let n = 2;

n *= 3 + 5; // el lado derecho es evaluado primero, es

alert( n ); // 16
```

Incremento/decremento

Aumentar o disminuir un número en uno es una de las operaciones numéricas más comunes.

Entonces, hay operadores especiales para ello:

- **Incremento** `++` incrementa una variable por 1:

```
let counter = 2;
counter++;      // funciona igual que counter = cour
alert( counter ); // 3
```

- **Decremento** `--` decrementa una variable por 1:

```
let counter = 2;
counter--;      // funciona igual que counter = cour
alert( counter ); // 1
```

> **Importante:**
>
> Incremento/decremento sólo puede ser aplicado a variables. Intentar utilizarlo en un valor como `5++` dará un error.

Los operadores `++` y `--` pueden ser colocados antes o después de una variable.

- Cuando el operador va después de la variable, está en "forma de sufijo": `counter++`.
- La "forma de prefijo" es cuando el operador va antes de la variable: `++counter`.

Ambas sentencias hacen la misma cosa: aumentar `counter` por `1`.

¿Existe alguna diferencia? Sí, pero solamente la podemos ver si utilizamos el valor devuelto de `++/--`.

Aclaremos. Tal como conocemos, todos los operadores devuelven un valor. Incremento/decremento no es una excepción. La forma prefijo devuelve el nuevo valor mientras que la forma sufijo devuelve el valor anterior (antes del incremento/decremento).

Para ver la diferencia, aquí hay un ejemplo:

```
let counter = 1;
let a = ++counter; // (*)

alert(a); // 2
```

En la línea `(*)`, la forma *prefijo* `++counter` incrementa `counter` y devuelve el nuevo valor, `2`. Por lo tanto, el `alert` muestra `2`.

Ahora usemos la forma sufijo:

```
let counter = 1;
let a = counter++; // (*) cambiado ++counter a counter

alert(a); // 1
```

En la línea `(*)`, la forma *sufijo* `counter++` también incrementa `counter`, pero devuelve el *antiguo* valor (antes de incrementar). Por lo tanto, el `alert` muestra `1`.

Para resumir:

- Si no se usa el resultado del incremento/decremento, no hay diferencia en la forma de usar:

```
let counter = 0;
counter++;
++counter;
alert( counter ); // 2, las líneas de arriba realiza
```

- Si queremos aumentar un valor *y* usar inmediatamente el resultado del operador, necesitamos la forma de prefijo:

```
let counter = 0;
alert( ++counter ); // 1
```

- Si queremos incrementar un valor, pero usamos su valor anterior, necesitamos la forma sufijo:

```
let counter = 0;
alert( counter++ ); // 0
```

Incremento/decremento entre otros operadores

Los operadores `++/--` también pueden ser usados dentro de expresiones. Su precedencia es más alta que la mayoría de los otros operadores aritméticos.

Por ejemplo:

```
let counter = 1;
alert( 2 * ++counter ); // 4
```

Compara con:

```
let counter = 1;
alert( 2 * counter++ ); // 2, porque counter++ c
```

Aunque técnicamente está bien, tal notación generalmente hace que el código sea menos legible. Una línea hace varias cosas, no es bueno.

Mientras lee el código, un rápido escaneo ocular "vertical" puede pasar por alto fácilmente algo como 'counter++' y no será obvio que la variable aumentó.

Aconsejamos un estilo de "una línea – una acción":

```
let counter = 1;
alert( 2 * counter );
counter++;
```

Operadores a nivel de bit

Los operadores a nivel bit tratan los argumentos como números enteros de 32 bits y trabajan en el nivel de su representación binaria.

Estos operadores no son específicos de JavaScript. Son compatibles con la mayoría de los lenguajes de programación.

La lista de operadores:

- AND (`&`)
- OR (`|`)
- XOR (`^`)
- NOT (`~`)
- LEFT SHIFT (`<<`)
- RIGHT SHIFT (`>>`)
- ZERO-FILL RIGHT SHIFT (`>>>`)

Estos operadores se usan muy raramente, cuando necesitamos manejar la representación de números en su más bajo nivel. No tenemos en vista usarlos pronto pues en el desarrollo web tiene poco uso; pero en ciertas áreas especiales, como la criptografía, son útiles. Puedes leer el artículo Operadores a nivel de bit en MDN cuando surja la necesidad.

Coma

El operador coma `,` es uno de los operadores más raros e inusuales. A veces se utiliza para escribir código más corto, por lo que debemos conocerlo para poder entender lo que está sucediendo.

El operador coma nos permite evaluar varias expresiones, separándolas con una coma `,`. Cada expresión se evalúa, pero sólo se devuelve el resultado de la última.

Por ejemplo:

```
let a = (1 + 2, 3 + 4);

alert( a ); // 7 (el resultado de 3 + 4)
```

Aquí, se evalúa la primera expresión `1 + 2` y se desecha su resultado. Luego, se evalúa `3 + 4` y se devuelve como resultado.

> **Coma tiene muy baja precedencia**
>
> Tenga en cuenta que el operador coma tiene una precedencia muy baja, inferior a `=`, por lo que los paréntesis son importantes en el ejemplo anterior.
>
> Sin ellos: `a = 1 + 2, 3 + 4` se evalúa primero el `+`, sumando los números a `a = 3, 7`, luego el operador de asignación `=` asigna `a = 3`, y el resto es ignorado. Es igual que `(a = 1 + 2), 3 + 4`.

¿Por qué necesitamos un operador que deseche todo excepto la última expresión?

A veces se utilizan en construcciones más complejas para ejecutar varias acciones en una línea.

Por ejemplo:

```
// tres operaciones en una línea
for (a = 1, b = 3, c = a * b; a < 10; a++) {
 ...
}
```

Tales trucos se usan en muchos frameworks de JavaScript, por eso los mencionamos. Sin embargo, generalmente no mejoran la legibilidad del código, por lo que debemos pensar bien antes de usarlos.

Tareas

Las formas sufijo y prefijo

importancia: 5

¿Cuáles son los valores finales de todas las variables `a`, `b`, `c` y `d` después del código a continuación?

```
let a = 1, b = 1;

let c = ++a; // ?
let d = b++; // ?
```

solución

Resultado de asignación

importancia: 3

¿Cuáles son los valores de 'a' y 'x' después del código a continuación?

```
let a = 2;

let x = 1 + (a *= 2);
```

solución

Conversiones de tipos

importancia: 5

¿Cuáles son los resultados de estas expresiones?

```
"" + 1 + 0
"" - 1 + 0
true + false
6 / "3"
"2" * "3"
4 + 5 + "px"
"$" + 4 + 5
"4" - 2
"4px" - 2
"  -9  " + 5
"  -9  " - 5
null + 1
undefined + 1
" \t \n" - 2
```

Piensa bien, anótalos y luego compara con la respuesta.

solución

Corregir la adición

importancia: 5

Aquí hay un código que le pide al usuario dos números y muestra su suma.

Funciona incorrectamente. El resultado en el ejemplo a continuación es `12` (para valores de captura predeterminados).

¿Por qué? Arreglalo. El resultado debería ser `3`.

```
let a = prompt("¿Primer número?", 1);
let b = prompt("¿Segundo número?", 2);

alert(a + b); // 12
```

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

9 de marzo de 2026

Comparaciones

Conocemos muchos operadores de comparación de las matemáticas:

En Javascript se escriben así:

- Mayor/menor que: `a > b`, `a < b`.
- Mayor/menor o igual que: `a >= b`, `a <= b`.
- Igual: `a == b` (ten en cuenta que el doble signo `==` significa comparación, mientras que un solo símbolo `a = b` significaría una asignación).
- Distinto. En matemáticas la notación es `≠`, pero en JavaScript se escribe como una asignación con un signo de exclamación delante: `a != b`.

En este artículo, aprenderemos más sobre los diferentes tipos de comparaciones y de cómo las realiza JavaScript, incluidas las peculiaridades importantes.

Al final, encontrará una buena receta para evitar problemas relacionadas con las "peculiaridades" de JavaScript.

Booleano es el resultado

Como todos los demás operadores, una comparación retorna un valor. En este caso, el valor es un booleano.

- `true` – significa "sí", "correcto" o "verdad".
- `false` – significa "no", "equivocado" o " no verdad".

Por ejemplo:

```
1 alert( 2 > 1 ); // true (correcto)
2 alert( 2 == 1 ); // false (incorrecto)
3 alert( 2 != 1 ); // true (correcto)
```

El resultado de una comparación puede asignarse a una variable, igual que cualquier valor:

```
1 let result = 5 > 4; // asignar el resultado de la comp
2 alert( result ); // true
```

Comparación de cadenas

Para ver si una cadena es "mayor" que otra, JavaScript utiliza el llamado orden "de diccionario" o "lexicográfico".

En otras palabras, las cadenas se comparan letra por letra.

Por ejemplo:

```
alert( 'Z' > 'A' ); // true
alert( 'Glow' > 'Glee' ); // true
alert( 'Bee' > 'Be' ); // true
```

El algoritmo para comparar dos cadenas es simple:

1. Compare el primer carácter de ambas cadenas.
2. Si el primer carácter de la primera cadena es mayor (o menor) que el de la otra cadena, entonces la primera cadena es mayor (o menor) que la segunda. Hemos terminado.
3. De lo contrario, si los primeros caracteres de ambas cadenas son los mismos, compare los segundos caracteres de la misma manera.
4. Repita hasta el final de cada cadena.
5. Si ambas cadenas tienen la misma longitud, entonces son iguales. De lo contrario, la cadena más larga es mayor.

En los ejemplos anteriores, la comparación `'Z' > 'A'` llega a un resultado en el primer paso.

La segunda comparación `"Glow"` y `"Glee"` necesitan más pasos, se comparan carácter por carácter:

1. `G` es igual que `G`.
2. `l` es igual que `l`.
3. `o` es mayor que `e`. Detente aquí. La primera cadena es mayor.

> **No es un diccionario real, sino un orden Unicode**
>
> El algoritmo de comparación dado arriba es aproximadamente equivalente al utilizado en los diccionarios o guías telefónicas, pero no es exactamente el mismo.
>
> Por ejemplo, las mayúsculas importan. Una letra mayúscula `"A"` no es igual a la minúscula `"a"`. ¿Cuál es mayor? La `"a"` minúscula. ¿Por qué? Porque el carácter en minúsculas tiene un mayor índice en la tabla de codificación interna que utiliza JavaScript (Unicode). Volveremos a los detalles específicos y las consecuencias de esto en el capítulo Strings.

Comparación de diferentes tipos

Al comparar valores de diferentes tipos, JavaScript convierte los valores a números.

Por ejemplo:

```
alert( '2' > 1 ); // true, la cadena '2' se convierte
alert( '01' == 1 ); // true, la cadena '01' se convier
```

Para valores booleanos, `true` se convierte en `1` y `false` en `0`.

Por ejemplo:

```
alert( true == 1 ); // true
alert( false == 0 ); // true
```

Una consecuencia graciosa

Es posible que al mismo tiempo:

- Dos valores sean iguales.
- Uno de ellos sea `true` como booleano y el otro sea `false` como booleano.

Por ejemplo:

```
let a = 0;
alert( Boolean(a) ); // false

let b = "0";
alert( Boolean(b) ); // true

alert( a == b ); // true!
```

Desde el punto de vista de JavaScript, este resultado es bastante normal. Una comparación de igualdad convierte valores utilizando la conversión numérica (de ahí que `"0"` se convierta en `0`), mientras que la conversión explícita `Boolean` utiliza otro conjunto de reglas.

Igualdad estricta

Una comparación regular de igualdad `==` tiene un problema. No puede diferenciar `0` de `falso':

```
alert( 0 == false ); // true
```

Lo mismo sucede con una cadena vacía:

```
alert( '' == false ); // true
```

Esto sucede porque los operandos de diferentes tipos son convertidos a números por el operador de igualdad `==`. Una cadena vacía, al igual que `false`, se convierte en un cero.

¿Qué hacer si queremos diferenciar `0` de `false`?

Un operador de igualdad estricto `===` comprueba la igualdad sin conversión de tipo.

En otras palabras, si `a` y `b` son de diferentes tipos, entonces `a === b` retorna inmediatamente `false` sin intentar convertirlos.

Intentémoslo:

```
alert( 0 === false ); // falso, porque los tipos son d
```

Existe también un operador de “diferencia estricta” `!==` análogo a `!=`.

El operador de igualdad estricta es un poco más largo de escribir, pero hace obvio lo que está pasando y deja menos espacio a errores.

Comparación con nulos e indefinidos

Veamos más casos extremos.

Hay un comportamiento no intuitivo cuando se compara `null` o `undefined` con otros valores.

Para un control de igualdad estricto `===`

Estos valores son diferentes, porque cada uno de ellos es de un tipo diferente.

```
alert( null === undefined ); // false
```

Para una comparación no estricta `==`

Hay una regla especial. Estos dos son una " pareja dulce ": son iguales entre sí (en el sentido de `==`), pero no a ningún otro valor.

```
alert( null == undefined ); // true
```

Para matemáticas y otras comparaciones `< > <= >=`

`null/undefined` se convierten en números: `null` se convierte en `0`, mientras que `undefined` se convierte en `NaN`.

Ahora veamos algunos hechos graciosos que suceden cuando aplicamos estas reglas. Y, lo que es más importante, cómo no caer en una trampa con ellas.

Resultado extraño: null vs 0

Comparemos `null` con un cero:

```
alert( null > 0 ); /// (1) false
alert( null == 0 ); /// (2) false
alert( null >= 0 ); // (3) true
```

Matemáticamente, eso es extraño. El último resultado afirma que “`null` es mayor o igual a cero”, así que en una de las comparaciones anteriores debe ser `true`, pero ambas son falsas.

La razón es que una comparación de igualdad `==` y las comparaciones `> < >= <=` funcionan de manera diferente. Las comparaciones convierten a `null` en un número, tratándolo como `0`. Es por eso que (3) `null >= 0` es verdadero y (1) `null > 0` es falso.

Por otro lado, el control de igualdad `==` para `undefined` y `null` se define de tal manera que, sin ninguna conversión, son iguales entre sí y no son iguales a

nada más. Es por eso que (2) `null == 0` es falso.

Un indefinido incomparable

El valor `undefined` no debe compararse con otros valores:

```
1 alert( undefined > 0 ); // false (1)
2 alert( undefined < 0 ); // false (2)
3 alert( undefined == 0 ); // false (3)
```

¿Por qué le desagrada tanto el cero? ¡Siempre falso!

Obtenemos estos resultados porque:

- Las comparaciones `(1)` y `(2)` retornan `falso` porque `no definido` se convierte en `NaN` y `NaN` es un valor numérico especial que retorna `falso` para todas las comparaciones.
- La comparación de igualdad `(3)` retorna `falso` porque `undefined` sólo equivale a `null` y a ningún otro valor.

Evitar los problemas

¿Por qué repasamos estos ejemplos? ¿Deberíamos recordar estas peculiaridades todo el tiempo? Bueno, en realidad no. De hecho, estas peculiaridades se volverán familiares con el tiempo, pero hay una manera sólida de evadir los problemas con ellas:

- Trata cualquier comparación con `undefined/null` (excepto la igualdad estricta `===`) con sumo cuidado.
- No uses comparaciones `>= > < <=` con una variable que puede ser `null/undefined`, a menos que estés realmente seguro de lo que estás haciendo. Si una variable puede tener estos valores, verifícalos por separado.

Resumen

- Los operadores de comparación devuelven un valor booleano.
- Las cadenas se comparan letra por letra, siguiendo el orden del diccionario.
- Cuando se comparan valores de distintos tipos, se convierten a números (excepto en la comprobación de igualdad estricta `===`).
- Los valores `null` y `undefined` son iguales `==` a sí mismos y entre sí, pero no equivalen a ningún otro valor.
- Ten cuidado al usar comparaciones como `>` o `<` con variables que ocasionalmente pueden ser `null` o `undefined`. Es recomendable revisar esos casos por separado.

Tareas

Comparaciones

importancia: 5

¿Cuál será el resultado de las siguientes expresiones?

```
5 > 4
"apple" > "pineapple"
"2" > "12"
undefined == null
undefined === null
null == "\n0\n"
null === +"\n0\n"
```

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

5 de julio de 2025

Ejecución condicional: if, '?'

A veces necesitamos que, bajo condiciones diferentes, se ejecuten acciones diferentes.

Para esto podemos usar la sentencia `if` y el "operador condicional" `?`.

La sentencia "if"

La sentencia `if(...)` evalúa la condición en los paréntesis, y si el resultado es verdadero (`true`), ejecuta un bloque de código.

Por ejemplo:

```
let year = prompt('¿En que año fué publicada la especi

if (year == 2015) alert( '¡Estás en lo cierto!' );
```

Aquí la condición es una simple igualdad (`year == 2015`), pero podría ser mucho más compleja.

Si queremos ejecutar más de una sentencia, debemos encerrar nuestro bloque de código entre llaves:

```
if (year == 2015) {
  alert( "¡Es Correcto!" );
  alert( "¡Eres muy inteligente!" );
}
```

Recomendamos encerrar nuestro bloque de código entre llaves `{}` siempre que se utilice la sentencia `if`, incluso si solo se va a ejecutar una sola sentencia. Al hacerlo mejoramos la legibilidad.

Conversión Booleana

La sentencia `if (…)` evalúa la expresión dentro de sus paréntesis y convierte el resultado en booleano.

Recordemos las reglas de conversión del capítulo Conversiones de Tipos:

- Los valores: número `0`, string vacío `""`, `null`, `undefined`, y `NaN`, en un contexto booleano se convierten en `false`. Por ello, sin ser booleanos, se los conoce como valores "falsy".
- El resto de los valores se convierten en `true`, y son valores que llamamos "truthy".

Entonces, el código bajo esta condición nunca se ejecutaría:

```
if (0) { // 0 es falsy
  ...
}
```

...y dentro de esta condición siempre se ejecutará:

```
if (1) { // 1 es truthy
  ...
}
```

También podemos pasar un valor booleano pre-evaluado al `if`, así:

```
let cond = (year == 2015); // la igualdad se evalúa y

if (cond) {
  ...
}
```

La cláusula “else”

La sentencia `if` puede contener un bloque `else` (“si no”, “en caso contrario”) opcional. Este bloque se ejecuta cuando la condición resulta falsa.

Por ejemplo:

```
let year = prompt('¿En qué año fue publicada la especi

if (year == 2015) {
  alert( '¡Lo adivinaste, correcto!' );
} else {
  alert( '¿Cómo puedes estar tan equivocado?' ); // cu
}
```

Muchas condiciones: “else if”

A veces queremos probar más de una condición. La clausula `else if` nos permite hacer esto.

Por ejemplo:

```
let year = prompt('¿En qué año fue publicada la especi

if (year < 2015) {
  alert( 'Muy poco...' );
} else if (year > 2015) {
  alert( 'Muy Tarde' );
} else {
  alert( '¡Exactamente!' );
}
```

En el código de arriba, JavaScript primero revisa si `year < 2015`. Si esto es falso, continúa a la siguiente condición `year > 2015`. Si esta también es falsa, mostrará la última `alert`.

Podría haber más bloques `else if`. Y el último `else` es opcional.

Operador ternario '?'

A veces necesitamos que el valor que asignemos a una variable dependa de alguna condición.

Por ejemplo:

```
let accessAllowed;
let age = prompt('¿Qué edad tienes?', '');

if (age > 18) {
  accessAllowed = true;
} else {
  accessAllowed = false;
}

alert(accessAllowed);
```

El "operador condicional" nos permite ejecutar esto en una forma más corta y simple.

El operador está representado por el signo de cierre de interrogación `?`. A veces es llamado "ternario" porque el operador tiene tres operandos, es el único operador de JavaScript que tiene esa cantidad.

La Sintaxis es:

```
let result = condition ? value1 : value2;
```

Se evalúa `condition`: si resulta verdadera entonces devuelve `value1`, de lo contrario `value2`.

Por ejemplo:

```
let accessAllowed = (age > 18) ? true : false;
```

Técnicamente, podemos omitir el paréntesis alrededor de `age > 18`. El operador de signo de interrogación tiene una precedencia baja, por lo que se

ejecuta después de la comparación `>`.

En este ejemplo realizaremos lo mismo que en el anterior:

```
// el operador de comparación  "age > 18" se ejecuta p
// (no necesitamos agregar los paréntesis)
let accessAllowed = age > 18 ? true : false;
```

Pero los paréntesis hacen el código mas legible, asi que recomendamos utilizarlos.

Por favor tome nota:

En el ejemplo de arriba, podrías evitar utilizar el operador de signo de interrogación porque esta comparación devuelve directamente `true/false`:

```
// es lo mismo que
let accessAllowed = age > 18;
```

Múltiples '?'

Una secuencia de operadores de signos de interrogación `?` puede devolver un valor que depende de más de una condición.

Por ejemplo:

```
let age = prompt('¿edad?', 18);

let message = (age < 3) ? '¡Hola, bebé!' :
  (age < 18) ? '¡Hola!' :
  (age < 100) ? '¡Felicidades!' :
  '¡Qué edad tan inusual!';

alert( message );
```

Puede ser difícil al principio comprender lo que está sucediendo. Pero después de una mirada más cercana, podemos ver que es solo una secuencia ordinaria de condiciones:

1. El primer signo de pregunta revisa si `age < 3`.
2. Si es cierto, devuelve `'¡Hola, bebé!'`. De lo contrario, continúa a la expresión que está después de los dos puntos ":", la cual revisa si `age < 18`.
3. Si es cierto, devuelve `'¡Hola!'`. De lo contrario, continúa con la expresión que está después de los dos puntos siguientes ":", la cual revisa si `age < 100`.
4. Si es cierto, devuelve `'¡Felicidades!'`. De lo contrario, continúa a la expresión que está después de los dos puntos ":", la cual devuelve `'¡Qué edad tan inusual!'`.

Aquí lo podemos ver utilizando `if..else`:

```
if (age < 3) {
  message = '¡Hola, bebé!';
} else if (age < 18) {
  message = '¡Hola!';
} else if (age < 100) {
  message = '¡Felicidades!';
} else {
  message = '¡Qué edad tan inusual!';
}
```

Uso no tradicional de '?'

A veces, el signo de interrogación de cierre `?` se utiliza para reemplazar un `if`:

```
let company = prompt('¿Qué compañía creó JavaScript?');

(company == 'Netscape') ?
   alert('¡Correcto!') : alert('Equivocado.');
```

Dependiendo de la condición `company == 'Netscape'`, se ejecutará la primera o la segunda expresión del operador `?` y se mostrará una alerta.

Aquí no asignamos el resultado a una variable. En vez de esto, ejecutamos diferentes códigos dependiendo de la condición.

No recomendamos el uso del operador de signo de interrogación de esta forma.

La notación es más corta que la sentencia equivalente con `if`, lo cual seduce a algunos programadores. Pero es menos legible.

Aquí está el mismo código utilizando la sentencia `if` para comparar:

```
let company = prompt('¿Cuál compañía creó JavaScript?');

if (company == 'Netscape') {
  alert('¡Correcto!');
} else {
  alert('Equivocado.');
}
```

Nuestros ojos leen el código verticalmente. Los bloques de código que se expanden múltiples lineas son mas fáciles de entender que los las instrucciones largas horizontales.

El propósito del operador de signo de interrogación `?` es para devolver un valor u otro dependiendo de su condición. Por favor utilízala para exactamente esto. Utiliza la sentencia `if` cuando necesites ejecutar código en ramas distintas.

Tareas

if (un string con cero)

importancia: 5

Se mostrará el `alert`?

```
if ("0") {
  alert( 'Hello' );
}
```

solución

El nombre de JavaScript

importancia: 2

Usando el constructor `if..else`, escribe el código que pregunta: '¿Cuál es el nombre "oficial" de JavaScript?'

Si el visitante escribe "ECMAScript", entonces muestra: "¡Correcto!", de lo contrario muestra: "¿No lo sabes? ¡ECMAScript!"

Demo en nueva ventana

solución

Muestra el signo

importancia: 2

Usando el constructor `if..else`, escribe un código que obtenga a través de un `prompt` un número y entonces muestre en un `alert`:

- `1`, si el valor es mayor que cero,
- `-1`, si es menor que cero,
- `0`, si es igual a cero.

En la tarea asumimos que siempre el usuario introduce un número.

Demo en nueva ventana

solución

Reescribe el 'if' como '?'

importancia: 5

Reescriba esta condición `if` usando el operador ternario `'?'`:

```
let result;

if (a + b < 4) {
  result = 'Debajo';
} else {
  result = 'Encima';
}
```

solución

Reescriba el 'if..else' con '?'

importancia: 5

Reescriba el `if..else` utilizando operadores ternarios múltiples `'?'`.

Para mejor legibilidad, se recomienda dividirlo en múltiples lineas de código.

```
let message;

if (login == 'Empleado') {
  message = 'Hola';
} else if (login == 'Director') {
  message = 'Felicidades';
} else if (login == '') {
  message = 'Sin sesión';
} else {
  message = '';
}
```

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor acerca del proyecto

contáctenos

13 de julio de 2025

Operadores Lógicos

Hay cuatro operadores lógicos en JavaScript: `||` (O), `&&` (Y), `!` (NO), `??` (Fusión de nulos). Aquí cubrimos los primeros tres, el operador `??` se verá en el siguiente artículo.

Aunque sean llamados lógicos, pueden ser aplicados a valores de cualquier tipo, no solo booleanos. El resultado también puede ser de cualquier tipo.

Veamos los detalles.

|| (OR)

El operador `OR` se representa con dos símbolos de linea vertical:

```
result = a || b;
```

En la programación clásica, el OR lógico esta pensado para manipular solo valores booleanos. Si cualquiera de sus argumentos es `true`, retorna `true`, de lo contrario retorna `false`.

En JavaScript, el operador es un poco más complicado y poderoso. Pero primero, veamos qué pasa con los valores booleanos.

Hay cuatro combinaciones lógicas posibles:

```
alert(true || true); // true (verdadero)
alert(false || true); // true
alert(true || false); // true
alert(false || false); // false (falso)
```

Como podemos ver, el resultado es siempre `true` excepto cuando ambos operandos son `false`.

Si un operando no es un booleano, se lo convierte a booleano para la evaluación.

Por ejemplo, el número `1` es tratado como `true`, el número `0` como `false`:

```
if (1 || 0) { // Funciona como if( true || false )
  alert("¡valor truthy!");
}
```

La mayoría de las veces, OR `||` es usado en una declaración `if` para probar si *alguna* de las condiciones dadas es `true`.

Por ejemplo:

```
let hour = 9;

if (hour < 10 || hour > 18) {
  alert( 'La oficina esta cerrada.' );
}
```

Podemos pasar mas condiciones:

```
let hour = 12;
let isWeekend = true;

if (hour < 10 || hour > 18 || isWeekend) {
  alert("La oficina esta cerrada."); // Es fin de sema
}
```

OR "||" encuentra el primer valor verdadero

La lógica descrita arriba es algo clásica. Ahora, mostremos las características "extra" de JavaScript.

El algoritmo extendido trabaja de la siguiente forma.

Dado múltiples valores aplicados al operador OR:

```
result = value1 || value2 || value3;
```

El operador OR `||` realiza lo siguiente:

- Evalúa los operandos de izquierda a derecha.
- Para cada operando, convierte el valor a booleano. Si el resultado es `true`, se detiene y retorna el valor original de ese operando.
- Si todos los operandos han sido evaluados (todos eran `false`), retorna el ultimo operando.

Un valor es retornado en su forma original, sin la conversión.

En otras palabras, una cadena de OR `"||"` devuelve el primer valor verdadero o el último si ningún verdadero es encontrado.

Por ejemplo:

```
alert(1 || 0); // 1 (1 es un valor verdadero)

alert(null || 1); // 1 (1 es el primer valor verdadero
alert(null || 0 || 1); // 1 (el primer valor verdadero

alert(undefined || null || 0); // 0 (todos son valores
```

Esto brinda varios usos interesantes comparados al "OR puro, clásico, de solo booleanos".

1. **Obtener el primer valor verdadero de una lista de variables o expresiones.**

Por ejemplo, tenemos las variables `firstName`, `lastName` y `nickName`, todas opcionales (pueden ser undefined o tener valores falsy).

Usemos OR `||` para elegir el que tiene los datos y mostrarlo (o anónimo si no hay nada configurado):

```
let firstName = "";
let lastName = "";
let nickName = "SuperCoder";

alert( firstName || lastName || nickName || "Anonymo
```

Si todas las variables fueran falsy, aparecería `"Anonymous"`.

2. **Evaluación del camino más corto.**

   Otra característica del operador OR || es la evaluación de "el camino más corto" o "cortocircuito".

   Esto significa que `||` procesa sus argumentos hasta que se alcanza el primer valor verdadero, y ese valor se devuelve inmediatamente sin siquiera tocar el otro argumento.

   La importancia de esta característica se vuelve obvia si un operando no es solo un valor sino una expresión con un efecto secundario, como una asignación de variable o una llamada a función.

   En el siguiente ejemplo, solo se imprime el segundo mensaje:

```
true || alert("not printed");
false || alert("printed");
```

   En la primera línea, el operador OR `||` detiene la evaluación inmediatamente después de ver que es verdadera, por lo que la alerta no se ejecuta.

   A veces se usa esta función para ejecutar comandos solo si la condición en la parte izquierda es falsy.

&& (AND)

El operador AND es representado con dos ampersands `&&`:

```
result = a && b;
```

En la programación clásica, AND retorna `true` si ambos operandos son valores verdaderos y `false` en cualquier otro caso.

```
alert(true && true); // true
alert(false && true); // false
alert(true && false); // false
alert(false && false); // false
```

Un ejemplo con `if`:

```
let hour = 12;
let minute = 30;

if (hour == 12 && minute == 30) {
  alert("La hora es 12:30");
}
```

Al igual que con OR, cualquier valor es permitido como operando de AND:

```
if (1 && 0) { // evaluado como true && false
  alert( "no funcionará porque el resultado es un valo
}
```

AND “&&” encuentra el primer valor falsy

Dado múltiples valores aplicados al operador AND:

```
result = value1 && value2 && value3;
```

El operador AND `&&` realiza lo siguiente:

- Evalúa los operandos de izquierda a derecha.
- Para cada operando, los convierte a un booleano. Si el resultado es `false`, se detiene y retorna el valor original de dicho operando.
- Si todos los operandos han sido evaluados (todos fueron valores truthy), retorna el último operando.

En otras palabras, AND retorna el primer valor falsy o el último valor si ninguno fue encontrado.

Las reglas anteriores son similares a las de OR. La diferencia es que AND retorna el primer valor *falsy* mientras que OR retorna el primer valor *truthy*.

Ejemplo:

```
// si el primer operando es un valor truthy,
// AND retorna el segundo operando:
alert(1 && 0); // 0
alert(1 && 5); // 5

// si el primer operando es un valor falsy,
// AND lo retorna. El segundo operando es ignorado
alert(null && 5); // null
alert(0 && "cualquier valor"); // 0
```

También podemos pasar varios valores de una vez. Observa como el primer valor falsy es retornado:

```
alert(1 && 2 && null && 3); // null
```

Cuando todos los valores son verdaderos, el último valor es retornado:

```
alert(1 && 2 && 3); // 3, el último.
```

La precedencia de AND `&&` es mayor que la de OR `||`

La precedencia del operador AND `&&` es mayor que la de OR `||`.

Así que el código `a && b || c && d` es básicamente el mismo que si la expresiones `&&` estuvieran entre paréntesis: `(a && b) || (c && d)`

No remplace *if* con || ni &&

A veces, la gente usa el operador AND `&&` como una "forma más corta de escribir `if`".

Por ejemplo:

```
let x = 1;

(x > 0) && alert("¡Mayor que cero!");
```

La acción en la parte derecha de `&&` sería ejecutada sólo si la evaluación la alcanza. Eso es, solo si `(x > 0)` es verdadero.

Así que básicamente tenemos un análogo para:

```
let x = 1;

if (x > 0) alert("Mayor que cero!");
```

Aunque la variante con `&&` parece más corta, `if` es más obvia y tiende a ser un poco más legible. Por lo tanto, recomendamos usar cada construcción para su propósito: use `if` si queremos si y use `&&` si queremos AND.

! (NOT)

El operador booleano NOT se representa con un signo de exclamación `!`.

La sintaxis es bastante simple:

```
result = !value;
```

El operador acepta un solo argumento y realiza lo siguiente:

1. Convierte el operando al tipo booleano: `true/false`.
2. Retorna el valor contrario.

Por ejemplo:

```
alert(!true); // false
alert(!0); // true
```

Un doble NOT `!!` es a veces usado para convertir un valor al tipo booleano:

```
alert(!!"cadena de texto no vacía"); // true
alert(!!null); // false
```

Eso es, el primer NOT convierte el valor a booleano y retorna el inverso, y el segundo NOT lo invierte de nuevo. Al final, tenemos una simple conversión a booleano.

Hay una manera un poco mas prolija de realizar lo mismo – una función integrada `Boolean`:

```
alert(Boolean("cadena de texto no vacía")); // true
alert(Boolean(null)); // false
```

La precedencia de NOT `!` es la mayor de todos los operadores lógicos, así que siempre se ejecuta primero, antes que `&&` o `||`.

Tareas

¿Cuál es el resultado de OR?

importancia: 5

¿Cuál será la salida del siguiente código?

```
alert( null || 2 || undefined );
```

solución

¿Cuál es el resultado de las alertas aplicadas al operador OR?

importancia: 3

¿Cuál será la salida del siguiente código?

```
alert( alert(1) || 2 || alert(3) );
```

solución

¿Cuál es el resultado de AND?

importancia: 5

¿Cuál será la salida del siguiente código?

```
alert( 1 && null && 2 );
```

solución

¿Cuál es el resultado de las alertas aplicadas al operador AND?

importancia: 3

¿Cuál será la salida del siguiente código?

```
alert( alert(1) && alert(2) );
```

solución

El resultado de OR AND OR

importancia: 5

¿Cuál será el resultado?

```
alert( null || 2 && 3 || 4 );
```

solución

Comprueba el rango por dentro

importancia: 3

Escribe una condición "if" para comprobar que `age` (edad) está entre `14` y `90` inclusive.

"Inclusive" significa que `age` puede llegar a ser uno de los extremos, `14` o `90`.

solución

Comprueba el rango por fuera

importancia: 3

Escribe una condición `if` para comprobar que `age` NO está entre 14 y 90 inclusive.

Crea dos variantes: la primera usando NOT `!`, y la segunda sin usarlo.

solución

Un pregunta acerca de "if"

importancia: 5

¿Cuáles de estos `alert` s va a ejecutarse?

¿Cuáles serán los resultados de las expresiones dentro de `if(...)`?

```
if (-1 || 0) alert( "primero" );
if (-1 && 0) alert( "segundo" );
if (null || -1 && 1) alert( "tercero" );
```

solución

Comprueba el inicio de sesión

importancia: 3

Escribe un código que pregunte por el inicio de sesión con `propmt`.

Si el visitante ingresa `"Admin"`, entonces debe pedir una contraseña (con un nuevo `propmt`). Si la entrada es una linea vacía o `Esc`, entonces muestra "Cancelado.". Si es otra cadena de texto, entonces muestra "No te conozco".

La contraseña se comprueba de la siguiente manera:

- Si es igual a "TheMaster", entonces muestra "Bienvenido!",
- Si es otra cadena de texto, muetra "Contraseña incorrecta",
- Para una cadena de texto vacía o una entrada cancelada, muestra "Cancelado."

El esquema:

Por favor usa bloques anidados de `if`. Piensa en la legibilidad general del código.

Pista: si se le pasa una entrada vacía a un prompt, retorna una cadena de texto vacía `''`. Presionando `ESC` durante un prompt retorna `null`.

Ejecutar el demo

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

14 de noviembre de 2022

Operador Nullish Coalescing '??'

Una adición reciente

Esta es una adición reciente al lenguaje. Los navegadores antiguos pueden necesitar polyfills.

El operador "nullish coalescing" (fusión de null) se escribe con un doble signo de cierre de interrogación `??`.

Como este trata a `null` y a `undefined` de forma similar, usaremos un término especial para este artículo. Diremos que una expresión es "definida" cuando no es `null` ni `undefined`.

El resultado de `a ?? b`:

- si `a` está "definida", será `a`,
- si `a` no está "definida", será `b`.

Es decir, `??` devuelve el primer argumento cuando este no es `null` ni `undefined`. En caso contrario, devuelve el segundo.

El operador "nullish coalescing" no es algo completamente nuevo. Es solamente una sintaxis agradable para obtener el primer valor "definido" de entre dos.

Podemos reescribir `result = a ?? b` usando los operadores que ya conocemos:

```
result = (a !== null && a !== undefined) ? a : b;
```

Ahora debería estar absolutamente claro lo que `??` hace. Veamos dónde podemos utilizarlo.

El uso típico de `??` es brindar un valor predeterminado.

Por ejemplo, aquí mostramos `user` si su valor está "definido" (que no es `null` ni `undefined`). De otro modo, muestra `Anonymous`:

```
let user;

alert(user ?? "Anonymous"); // Anonymous (user es unde
```

Aquí el ejemplo de `user` con un nombre asignado:

```
let user = "John";

alert(user ?? "Anonymous"); // John (user no es null n
```

También podemos usar una secuencia de `??` para seleccionar el primer valor que no sea `null/undefined` de una lista.

Digamos que tenemos los datos de un usuario en las variables `firstName`, `lastName` y `nickName`. Todos ellos podrían ser indefinidos si el usuario decide no ingresar los valores correspondientes.

Queremos mostrar un nombre usando una de estas variables, o mostrar "anónimo" si todas ellas son `null/undefined`.

Usemos el operador `??` para ello:

```
let firstName = null;
let lastName = null;
let nickName = "Supercoder";

// Muestra el primer valor definido:
alert(firstName ?? lastName ?? nickName ?? "Anonymous"
```

Comparación con ||

El operador OR `||` puede ser usado de la misma manera que `??`, tal como está explicado en el capítulo previo

Por ejemplo, en el código de arriba podemos reemplazar `??` por `||` y obtener el mismo resultado:

```
let firstName = null;
let lastName = null;
let nickName = "Supercoder";

// muestra el primer valor "verdadero":
alert(firstName || lastName || nickName || "Anonymous"
```

Históricamente, el operador OR `||` estuvo primero. Existe desde el origen de JavaScript, así que los desarrolladores lo estuvieron usando para tal propósito durante mucho tiempo.

Por otro lado, el operador "nullish coalescing" `??` fue una adición reciente, y la razón es que la gente no estaba del todo satisfecha con `||`.

La gran diferencia es que:

- `||` devuelve el primer valor *verdadero*.
- `??` devuelve el primer valor *definido*.

El `||` no distingue entre `false`, `0`, un string vacío `""`, y `null/undefined`. Todos son lo mismo: valores "falsos". Si cualquiera de ellos es el primer argumento de `||`, obtendremos el segundo argumento como resultado.

Pero en la práctica podemos querer usar el valor predeterminado solamente cuando la variable es `null/undefined`, es decir cuando el valor realmente es desconocido o no fue establecido.

Por ejemplo considera esto:

```
let height = 0; // altura cero

alert(height || 100); // 100
alert(height ?? 100); // 0
```

`height || 100` verifica si `height` es "falso", y `0` lo es. - así el resultado de `||` es el segundo argumento, `100`. `height ?? 100` verifica si `height` es `null/undefined`, y no lo es. - así el resultado es `height` como está, que es `0`.

En la práctica, una altura cero es a menudo un valor válido que no debería ser reemplazado por un valor por defecto. En este caso `??` hace lo correcto.

Precedencia

La precedencia del operador `??` es la misma de `||`. Ambos son iguales a `3` en la Tabla MDN.

Esto significa que ambos operadores, `||` y `??`, son evaluados antes que `=` y `?`, pero después de la mayoría de las demás operaciones como `+` y `*`.

Así que podemos necesitar añadir paréntesis:

```
let height = null;
let width = null;

// Importante: usar paréntesis
let area = (height ?? 100) * (width ?? 50);

alert(area); // 5000
```

Caso contrario, si omitimos los paréntesis, entonces `*` tiene una mayor precedencia y se ejecutará primero. Eso sería lo mismo que:

```
// sin paréntesis
let area = height ?? 100 * width ?? 50;

// ...funciona de esta forma (no es lo que queremos):
let area = height ?? (100 * width) ?? 50;
```

Uso de ?? con && y ||

Por motivos de seguridad, JavaScript prohíbe el uso de `??` junto con los operadores `&&` y `||`, salvo que la precedencia sea explícitamente especificada con paréntesis.

El siguiente código desencadena un error de sintaxis:

```
let x = 1 && 2 ?? 3; // Syntax error
```

La limitación es debatible. Fue agregada a la especificación del lenguaje con propósito de evitar equivocaciones cuando la gente comenzara a reemplazar `||` por `??`.

Usa paréntesis explícitos para solucionarlo:

```
let x = (1 && 2) ?? 3; // Funciona

alert(x); // 2
```

Resumen

- El operador "nullish coalescing" `??` brinda una manera concisa de seleccionar un valor "definido" de una lista.

  Es usado para asignar valores por defecto a las variables:

  ```
  // Asignar height=100, si height es null o undefinec
  height = height ?? 100;
  ```

- El operador `??` tiene una precedencia muy baja, un poco más alta que `?` y `=`.
- Está prohibido su uso con `||` y `&&` sin paréntesis explícitos.

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor acerca del proyecto
contáctenos

26 de junio de 2022

Bucles: while y for

Usualmente necesitamos repetir acciones.

Por ejemplo, mostrar los elementos de una lista uno tras otro o simplemente ejecutar el mismo código para cada número del 1 al 10.

Los *Bucles* son una forma de repetir el mismo código varias veces.

> **Los bucles for...of y for...in**
>
> Un pequeño anuncio para lectores avanzados.
>
> Este artículo cubre solamente los bucles básicos: `while`, `do..while` y `for(..;..;..)`.
>
> Si llegó a este artículo buscando otro tipo de bucles, aquí están los enlaces:
>
> - Vea for...in para bucles sobre propiedades de objetos.
> - Vea for...of e iterables para bucles sobre arrays y objetos iterables.
>
> De otra manera, por favor continúe leyendo.

El bucle “while”

El bucle `while` (mientras) tiene la siguiente sintaxis:

```
while (condition) {
  // código
  // llamado "cuerpo del bucle"
}
```

Mientras la condición `condition` sea verdadera, el `código` del cuerpo del bucle será ejecutado.

Por ejemplo, el bucle debajo imprime `i` mientras se cumpla `i < 3`:

```
let i = 0;
while (i < 3) { // muestra 0, luego 1, luego 2
  alert( i );
  i++;
}
```

Cada ejecución del cuerpo del bucle se llama *iteración*. El bucle en el ejemplo de arriba realiza 3 iteraciones.

Si faltara `i++` en el ejemplo de arriba, el bucle sería repetido (en teoría) eternamente. En la práctica, el navegador tiene maneras de detener tales bucles desmedidos; y en el JavaScript del lado del servidor, podemos eliminar el proceso.

Cualquier expresión o variable puede usarse como condición del bucle, no solo las comparaciones: El `while` evaluará y transformará la condición a un booleano.

Por ejemplo, una manera más corta de escribir `while (i != 0)` es `while (i)`:

```
let i = 3;
while (i) { // cuando i sea 0, la condición se volverá
  alert( i );
  i--;
}
```

Las llaves no son requeridas para un cuerpo de una sola línea

Si el cuerpo del bucle tiene una sola sentencia, podemos omitir las llaves `{…}`:

```
let i = 3;
while (i) alert(i--);
```

El bucle “do…while”

La comprobación de la condición puede ser movida *debajo* del cuerpo del bucle usando la sintaxis `do..while`:

```
do {
  // cuerpo del bucle
} while (condition);
```

El bucle primero ejecuta el cuerpo, luego comprueba la condición, y, mientras sea un valor verdadero, la ejecuta una y otra vez.

Por ejemplo:

```
let i = 0;
do {
  alert( i );
  i++;
} while (i < 3);
```

Esta sintaxis solo debe ser usada cuando quieres que el cuerpo del bucle sea ejecutado **al menos una vez** sin importar que la condición sea verdadera. Usualmente, se prefiere la otra forma: `while(…) {…}`.

El bucle “for”

El bucle `for` es más complejo, pero también el más usado.

Se ve así:

```
for (begin; condition; step) { // (comienzo, condición
  // ... cuerpo del bucle ...
}
```

Aprendamos el significado de cada parte con un ejemplo. El bucle debajo corre `alert(i)` para `i` desde `0` hasta (pero no incluyendo) `3`:

```
for (let i = 0; i < 3; i++) { // muestra 0, luego 1, 1
  alert(i);
}
```

Vamos a examinar la declaración `for` parte por parte:

parte		
comienzo	`let i = 0`	Se ejecuta una vez al comienzo del bucle.
condición	`i < 3`	Comprobada antes de cada iteración del bucle. Si es falsa, el bucle finaliza.
cuerpo	`alert(i)`	Se ejecuta una y otra vez mientras la condición sea verdadera.
paso	`i++`	Se ejecuta después del cuerpo en cada iteración.

El algoritmo general del bucle funciona de esta forma:

```
Se ejecuta comenzar
→ (si condición → ejecutar cuerpo y ejecutar paso)
→ (si condición → ejecutar cuerpo y ejecutar paso)
→ (si condición → ejecutar cuerpo y ejecutar paso)
→ ...
```

Si eres nuevo en bucles, te podría ayudar regresar al ejemplo y reproducir cómo se ejecuta paso por paso en una pedazo de papel.

Esto es lo que sucede exactamente en nuestro caso:

```
// for (let i = 0; i < 3; i++) alert(i)

// se ejecuta comenzar
let i = 0
// si condición → ejecutar cuerpo y ejecutar paso
if (i < 3) { alert(i); i++ }
// si condición → ejecutar cuerpo y ejecutar paso
if (i < 3) { alert(i); i++ }
// si condición → ejecutar cuerpo y ejecutar paso
```

```
if (i < 3) { alert(i); i++ }
// ...finaliza, porque ahora i == 3
```

Declaración de variable en línea

Aquí, la variable "counter" `i` es declarada en el bucle. Esto es llamado una declaración de variable "en línea". Dichas variables son visibles solo dentro del bucle.

```
for (let i = 0; i < 3; i++) {
  alert(i); // 0, 1, 2
}
alert(i); // error, no existe dicha variable
```

En vez de definir una variable, podemos usar una que ya exista:

```
let i = 0;

for (i = 0; i < 3; i++) { // usa una variable ex
  alert(i); // 0, 1, 2
}

alert(i); // 3, visible, porque fue declarada fu
```

Omitiendo partes

Cualquier parte de `for` puede ser omitida.

Por ejemplo, podemos quitar `comienzo` si no necesitamos realizar nada al inicio del bucle.

Como aquí:

```
let i = 0; // Ya tenemos i declarada y asignada

for (; i < 3; i++) { // no hay necesidad de "comenzar"
  alert( i ); // 0, 1, 2
}
```

También podemos eliminar la parte `paso`:

```
let i = 0;

for (; i < 3;) {
  alert( i++ );
}
```

Esto hace al bucle idéntico a `while (i < 3)`.

En realidad podemos eliminar todo, creando un bucle infinito:

```
for (;;) {
  // se repite sin limites
}
```

Por favor, nota que los dos punto y coma `;` del `for` deben estar presentes. De otra manera, habría un error de sintaxis.

Rompiendo el bucle

Normalmente, se sale de un bucle cuando la condición se vuelve falsa.

Pero podemos forzar una salida en cualquier momento usando la directiva especial `break`.

Por ejemplo, el bucle debajo le pide al usuario por una serie de números, "rompiéndolo" cuando un número no es ingresado:

```
let sum = 0;

while (true) {

  let value = +prompt("Ingresa un número", '');

  if (!value) break; // (*)

  sum += value;

}
alert( 'Suma: ' + sum );
```

La directiva `break` es activada en la línea `(*)` si el usuario ingresa una línea vacía o cancela la entrada. Detiene inmediatamente el bucle, pasando el control a la primera línea después de el bucle. En este caso, `alert`.

La combinación "bucle infinito + `break` según sea necesario" es ideal en situaciones donde la condición del bucle debe ser comprobada no al inicio o al final de el bucle, sino a la mitad o incluso en varias partes del cuerpo.

Continuar a la siguiente iteración

La directiva `continue` es una "versión más ligera" de `break`. No detiene el bucle completo. En su lugar, detiene la iteración actual y fuerza al bucle a comenzar una nueva (si la condición lo permite).

Podemos usarlo si hemos terminado con la iteración actual y nos gustaría movernos a la siguiente.

El bucle debajo usa `continue` para mostrar solo valores impares:

```
for (let i = 0; i < 10; i++) {

  // si es verdadero, saltar el resto del cuerpo
  if (i % 2 == 0) continue;


```

```
  alert(i); // 1, luego 3, 5, 7, 9
}
```

Para los valores pares de `i`, la directiva `continue` deja de ejecutar el cuerpo y pasa el control a la siguiente iteración de `for` (con el siguiente número). Así que el `alert` solo es llamado para valores impares.

La directiva `continue` ayuda a disminuir la anidación

Un bucle que muestra valores impares podría verse así:

```
for (let i = 0; i < 10; i++) {

  if (i % 2) {
    alert( i );
  }

}
```

Desde un punto de vista técnico, esto es idéntico al ejemplo de arriba. Claro, podemos simplemente envolver el código en un bloque `if` en vez de usar `continue`.

Pero como efecto secundario, esto crearía un nivel más de anidación (la llamada a `alert` dentro de las llaves). Si el código dentro de `if` posee varias líneas, eso podría reducir la legibilidad en general.

No `break/continue` a la derecha de '?'

Por favor, nota que las construcciones sintácticas que no son expresiones no pueden user usadas con el operador ternario `?`. En particular, directivas como `break/continue` no son permitidas aquí.

Por ejemplo, si tomamos este código:

```
if (i > 5) {
  alert(i);
} else {
  continue;
}
```

...y lo reescribimos usando un signo de interrogación:

```
(i > 5) ? alert(i) : continue; // continue no e
```

...deja de funcionar. Código como este generará un error de sintaxis:

Esta es otra razón por la cual se recomienda no usar el operador de signo de interrogación `?` en lugar de `if`.

Etiquetas para break/continue

A veces necesitamos salirnos de múltiples bucles anidados al mismo tiempo.

Por ejemplo, en el código debajo usamos un bucle sobre `i` y `j`, solicitando las coordenadas `(i,j)` de `(0,0)` a `(3,3)`:

```
for (let i = 0; i < 3; i++) {

  for (let j = 0; j < 3; j++) {

    let input = prompt(`Valor en las coordenadas (${i}

    // ¿Y si quiero salir de aquí hacia Listo (debajo)

  }
}

alert('Listo!');
```

Necesitamos una manera de detener el proceso si el usuario cancela la entrada.

El `break` ordinario después de `input` solo nos sacaría del bucle interno. Eso no es suficiente. ¡Etiquetas, vengan al rescate!

Una *etiqueta* es un identificador con un signo de dos puntos ":" antes de un bucle:

```
labelName: for (...) {
  ...
}
```

La declaración `break <labelName>` en el bucle debajo nos saca hacia la etiqueta:

```
outer: for (let i = 0; i < 3; i++) {

  for (let j = 0; j < 3; j++) {

    let input = prompt(`Value at coords (${i},${j})`,

    // Si es una cadena de texto vacía o se canceló, e
    if (!input) break outer; // (*)

    // hacer algo con el valor...
  }
}

alert('Listo!');
```

En el código de arriba, `break outer` mira hacia arriba por la etiqueta llamada `outer` y nos saca de dicho bucle.

Así que el control va directamente de `(*)` a `alert('Listo!')`.

También podemos mover la etiqueta a una línea separada:

```
outer:
for (let i = 0; i < 3; i++) { ... }
```

La directiva `continue` también puede usar usada con una etiqueta. En este caso, la ejecución del código salta a la siguiente iteración del bucle etiquetado.

> **Las etiquetas no son "goto"**
>
> Las etiquetas no nos permiten saltar a un lugar arbitrario en el código.
>
> Por ejemplo, es imposible hacer esto:
>
> ```
> break label;  // ¿saltar a label? No funciona.
>
> label: for (...)
> ```
>
> Una directiva `break` debe estar en el interior del bucle. Aunque, técnicamente, puede estar en cualquier bloque de código etiquetado:
>
> ```
> label: {
>   // ...
>   break label; // funciona
>   // ...
> }
> ```
>
> ...Aunque 99.9% del tiempo `break` se usa dentro de bucles, como hemos visto en ejemplos previos.
>
> Un `continue` es solo posible dentro de un bucle.

Resumen

Cubrimos 3 tipos de bucles:

- `while` – La condición es comprobada antes de cada iteración.
- `do..while` – La condición es comprobada después de cada iteración.
- `for (;;)` – La condición es comprobada antes de cada iteración, con ajustes adicionales disponibles.

Para crear un bucle "infinito", usualmente se usa `while(true)`. Un bucle como este, tal y como cualquier otro, puede ser detenido con la directiva `break`.

Si queremos detener la iteración actual y adelantarnos a la siguiente, podemos usar la directiva `continue`.

`break/continue` soportan etiquetas antes del bucle. Una etiqueta es la única forma de usar `break/continue` para escapar de un bucle anidado para ir a uno exterior.

Tareas

Último valor del bucle

importancia: 3

¿Cuál es el último valor mostrado en alerta por este código? ¿Por qué?

```
let i = 3;

while (i) {
  alert( i-- );
}
```

solución

¿Qué valores serán mostrados por el bucle while?

importancia: 4

Para cada iteración del bucle, escribe qué valor será impreso y luego compáralo con la solución.

Ambos bucles ¿ `alertan` los mismos valores?

1.

La forma de prefijo `++i`:

```
let i = 0;
while (++i < 5) alert( i );
```

2.

La forma de sufijo `i++`

```
let i = 0;
while (i++ < 5) alert( i );
```

solución

¿Qué valores serán mostrados por el bucle "for"?

importancia: 4

Para cada bucle, anota qué valores mostrará y luego compara las respuestas.

Ambos bucles, ¿muestran en `alert` los mismos valores?

1.

La forma del sufijo:

```
for (let i = 0; i < 5; i++) alert( i );
```

2.

La forma del prefijo:

```
for (let i = 0; i < 5; ++i) alert( i );
```

solución

Muestra números pares en el bucle

importancia: 5

Usa el bucle `for` para mostrar números pares del `2` al `10`.

Ejecutar el demo

solución

Reemplaza "for" por "while"

importancia: 5

Reescribe el código cambiando el bucle `for` a `while` sin alterar su comportamiento (la salida debería ser la misma).

```
for (let i = 0; i < 3; i++) {
  alert( `número ${i}!` );
}
```

solución

Repite hasta que la entrada sea correcta

importancia: 5

Escribe un bucle que solicite un número mayor que `100`. Si el usuario ingresa otro número – pídele que ingrese un valor de nuevo.

El bucle debe pedir un número hasta que el usuario ingrese un número mayor que `100` o bien cancele la entrada/ingrese una linea vacía.

Aquí podemos asumir que el usuario solo ingresará números. No hay necesidad de implementar un manejo especial para entradas no numéricas en esta tarea.

Ejecutar el demo

solución

Muestra números primos

importancia: 3

Un número entero mayor que `1` es llamado primo si no puede ser dividido sin un resto por ningún número excepto `1` y él mismo.

En otras palabras, `n > 1` es un primo si no puede ser divido exactamente por ningún número excepto `1` y `n`.

Por ejemplo, `5` es un primo, porque no puede ser divido exactamente por `2`, `3` y `4`.

Escribe el código que muestre números primos en el intervalo de `2` a `n`.

Para `n = 10` el resultado será `2, 3, 5, 7`.

PD. El código debería funcionar para cualquier `n`, no debe estar programado para valores fijos.

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

6 de mayo de 2022

La sentencia "switch"

Una sentencia `switch` puede reemplazar múltiples condiciones `if`.

Provee una mejor manera de comparar un valor con múltiples variantes.

La sintaxis

`switch` tiene uno o mas bloques `case` y un opcional `default`.

Se ve de esta forma:

```
switch(x) {
  case 'valor1':  // if (x === 'valor1')
    ...
    [break]

  case 'valor2':  // if (x === 'valor2')
    ...
    [break]

  default:
    ...
    [break]
}
```

- El valor de `x` es comparado contra el valor del primer `case` (en este caso, `valor1`), luego contra el segundo (`valor2`) y así sucesivamente, todo esto bajo una igualdad estricta.
- Si la igualdad es encontrada, `switch` empieza a ejecutar el código iniciando por el primer `case` correspondiente, hasta el `break` más cercano (o hasta el final del `switch`).
- Si no se cumple ningún caso entonces el código `default` es ejecutado (si existe).

Ejemplo

Un ejemplo de `switch` (se resalta el código ejecutado):

```
let a = 2 + 2;

switch (a) {
  case 3:
    alert( 'Muy pequeño' );
    break;
  case 4:
    alert( '¡Exacto!' );
    break;
```

```
  case 5:
    alert( 'Muy grande' );
    break;
  default:
    alert( "Desconozco estos valores" );
}
```

Aquí el `switch` inicia comparando `a` con la primera variante `case` que es `3`. La comparación falla.

Luego `4`. La comparación es exitosa, por tanto la ejecución empieza desde `case 4` hasta el `break` más cercano.

Si no existe `break` entonces la ejecución continúa con el próximo `case` sin ninguna revisión.

Un ejemplo sin `break`:

```
let a = 2 + 2;

switch (a) {
  case 3:
    alert( 'Muy pequeño' );
  case 4:
    alert( '¡Exacto!' );
  case 5:
    alert( 'Muy grande' );
  default:
    alert( "Desconozco estos valores" );
}
```

En el ejemplo anterior veremos ejecuciones de tres `alert` secuenciales:

```
alert( '¡Exacto!' );
alert( 'Muy grande' );
alert( "Desconozco estos valores" );
```

Cualquier expresión puede ser un argumento `switch/case`

Ambos `switch` y `case` permiten expresiones arbitrarias.

Por ejemplo:

```
let a = "1";
let b = 0;

switch (+a) {
  case b + 1:
    alert("esto se ejecuta, porque +a es 1, exac
    break;

  default:
    alert("esto no se ejecuta");
}
```

Aquí `+a` da `1`, esto es comparado con `b + 1` en `case`, y el código correspondiente es ejecutado.

Agrupamiento de "case"

Varias variantes de `case` los cuales comparten el mismo código pueden ser agrupadas.

Por ejemplo, si queremos que se ejecute el mismo código para `case 3` y `case 5`:

```
let a = 2 + 2;

switch (a) {
  case 4:
    alert('¡Correcto!');
    break;

  case 3:                    // (*) agrupando dos case
  case 5:
    alert('¡Incorrecto!');
    alert("¿Por qué no tomas una clase de matemáticas?
    break;

  default:
    alert('El resultado es extraño. Realmente.');
}
```

Ahora ambos, `3` y `5`, muestran el mismo mensaje.

La capacidad de "agrupar" los `case` es un efecto secundario de cómo trabaja `switch/case` sin `break`. Aquí la ejecución de `case 3` inicia desde la línea `(*)` y continúa a través de `case 5`, porque no existe `break`.

El tipo importa

Vamos a enfatizar que la comparación de igualdad es siempre estricta. Los valores deben ser del mismo tipo para coincidir.

Por ejemplo, consideremos el código:

```
let arg = prompt("Ingrese un valor");
switch (arg) {
  case '0':
  case '1':
    alert( 'Uno o cero' );
    break;

  case '2':
    alert( 'Dos' );
    break;

  case 3:
    alert( '¡Nunca ejecuta!' );
    break;
  default:
    alert( 'Un valor desconocido' );
}
```

1. Para `0`, `1`, se ejecuta el primer `alert`.
2. Para `2` se ejecuta el segundo `alert`.
3. Pero para `3`, el resultado del `prompt` es un string `"3"`, el cual no es estrictamente igual `===` al número `3`. Por tanto ¡Tenemos un código muerto en `case 3`! La variante `default` se ejecutará.

Tareas

Reescribe el "switch" en un "if"

importancia: 5

Escribe el código utilizando `if..else` que corresponda al siguiente `switch`:

```
switch (navegador) {
  case 'Edge':
    alert( "¡Tienes Edge!" );
    break;

  case 'Chrome':
  case 'Firefox':
  case 'Safari':
  case 'Opera':
    alert( 'Esta bien, soportamos estos navegadores ta
    break;

  default:
    alert( '¡Esperamos que esta página se vea bien!' )
}
```

solución

Reescribe "if" en "switch"

importancia: 4

Reescribe el código debajo utilizando solo un argumento `switch`:

```
let a = +prompt('a?', '');

if (a == 0) {
  alert( 0 );
}
if (a == 1) {
  alert( 1 );
}

if (a == 2 || a == 3) {
  alert( '2,3' );
}
```

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

2 de agosto de 2026

Funciones

Muy a menudo necesitamos realizar acciones similares en muchos lugares del script.

Por ejemplo, debemos mostrar un mensaje atractivo cuando un visitante inicia sesión, cierra sesión y tal vez en otros momentos.

Las funciones son los principales "bloques de construcción" del programa. Permiten que el código se llame muchas veces sin repetición.

Ya hemos visto ejemplos de funciones integradas, como `alert(message)`, `prompt(message, default)` y `confirm(question)`. Pero también podemos crear funciones propias.

Declaración de funciones

Para crear una función podemos usar una *declaración de función*.

Se ve como aquí:

```
function showMessage() {
  alert( '¡Hola a todos!' );
}
```

La palabra clave `function` va primero, luego va el *nombre de función*, luego una lista de *parámetros* entre paréntesis (separados por comas, vacía en el ejemplo anterior) y finalmente el código de la función entre llaves, también llamado "el cuerpo de la función".

```
function name(parameter1, parameter2, ... parameterN)
 // body
}
```

Nuestra nueva función puede ser llamada por su nombre: `showMessage()`.

Por ejemplo:

```
function showMessage() {
  alert( '¡Hola a todos!' );
}

showMessage();
showMessage();
```

La llamada `showMessage()` ejecuta el código de la función. Aquí veremos el mensaje dos veces.

Este ejemplo demuestra claramente uno de los propósitos principales de las funciones: evitar la duplicación de código...

Si alguna vez necesitamos cambiar el mensaje o la forma en que se muestra, es suficiente modificar el código en un lugar: la función que lo genera.

Variables Locales

Una variable declarada dentro de una función solo es visible dentro de esa función.

Por ejemplo:

```
function showMessage() {
  let message = "Hola, ¡Soy JavaScript!"; // variable

  alert( message );
}

showMessage(); // Hola, ¡Soy JavaScript!

alert( message ); // <-- ¡Error! La variable es local
```

Variables Externas

Una función también puede acceder a una variable externa, por ejemplo:

```
let userName = 'Juan';

function showMessage() {
  let message = 'Hola, ' + userName;
  alert(message);
}

showMessage(); // Hola, Juan
```

La función tiene acceso completo a la variable externa. Puede modificarlo también.

Por ejemplo:

```
let userName = 'Juan';

function showMessage() {
  userName = "Bob"; // (1) Cambió la variable externa

  let message = 'Hola, ' + userName;
  alert(message);
}

alert( userName ); // Juan antes de llamar la función

showMessage();
```

```
alert( userName ); // Bob, el valor fué modificado por
```

La variable externa solo se usa si no hay una local.

Si una variable con el mismo nombre se declara dentro de la función, le *hace sombra* a la externa. Por ejemplo, en el siguiente código, la función usa la variable `userName` local. La exterior se ignora:

```
let userName = 'John';

function showMessage() {
  let userName = "Bob"; // declara variable local

  let message = 'Hello, ' + userName; // Bob
  alert(message);
}

// la función crea y utiliza su propia variable local
showMessage();

alert( userName ); // John, se mantiene, la función no
```

Variables globales

Variables declaradas fuera de cualquier función, como la variable externa `userName` en el código anterior, se llaman *globales*.

Las variables globales son visibles desde cualquier función (a menos que se les superpongan variables locales con el mismo nombre).

Es una buena práctica reducir el uso de variables globales. El código moderno tiene pocas o ninguna variable global. La mayoría de las variables residen en sus funciones. Aunque a veces puede justificarse almacenar algunos datos a nivel de proyecto.

Parámetros

Podemos pasar datos arbitrarios a funciones usando parámetros.

En el siguiente ejemplo, la función tiene dos parámetros: `from` y `text`.

```
function showMessage(from, text) { // parámetros: from
  alert(from + ': ' + text);
}

showMessage('Ann', '¡Hola!'); // Ann: ¡Hola! (*)
showMessage('Ann', "¿Cómo estás?"); // Ann: ¿Cómo está
```

Cuando la función se llama `(*)` y `(**)`, los valores dados se copian en variables locales `from` y `text`. Y la función las utiliza.

Aquí hay un ejemplo más: tenemos una variable `from` y la pasamos a la función. Tenga en cuenta: la función cambia `from`, pero el cambio no se ve afuera, porque una función siempre obtiene una copia del valor:

```
function showMessage(from, text) {

  from = '*' + from + '*'; // hace que "from" se vea m

  alert( from + ': ' + text );
}

let from = "Ann";

showMessage(from, "Hola"); // *Ann*: Hola

// el valor de "from" es el mismo, la función modificó
alert( from ); // Ann
```

Cuando un valor es pasado como un parámetro de función, también se denomina *argumento*.

Para poner los términos claros:

- Un parámetro es una variable listada dentro de los paréntesis en la declaración de función (es un término para el momento de la declaración)
- Un argumento es el valor que es pasado a la función cuando esta es llamada (es el término para el momento en que se llama).

Declaramos funciones listando sus parámetros, luego las llamamos pasándoles argumentos.

En el ejemplo de arriba, se puede decir: "la función `showMessage` es declarada con dos parámetros, y luego llamada con dos argumentos: `from` y `"Hola"`".

Valores predeterminados

Si una función es llamada, pero no se le proporciona un argumento, su valor correspondiente se convierte en `undefined`.

Por ejemplo, la función mencionada anteriormente `showMessage(from, text)` se puede llamar con un solo argumento:

```
showMessage("Ann");
```

Eso no es un error. La llamada mostraría `"Ann: undefined"`. Como no se pasa un valor de `text`, este se vuelve `undefined`.

Podemos especificar un valor llamado "predeterminado" o "por defecto" (es el valor que se usa si el argumento fue omitido) en la declaración de función usando `=`:

```
function showMessage(from, text = "sin texto") {
  alert( from + ": " + text );
}


```

```
showMessage("Ann"); // Ann: sin texto
```

Ahora, si no se pasa el parámetro `text`, obtendrá el valor `"sin texto"`

El valor predeterminado también se asigna si el parámetro existe pero es estrictamente igual a `undefined`:

```
1 showMessage("Ann", undefined); // Ann: sin texto
```

Aquí `"sin texto"` es un string, pero puede ser una expresión más compleja, la cual solo es evaluada y asignada si el parámetro falta. Entonces, esto también es posible:

```
1 function showMessage(from, text = anotherFunction()) {
2   // anotherFunction() solo se ejecuta si text no fue
3   // su resultado se convierte en el valor de texto
4 }
```

Evaluación de parámetros predeterminados

En JavaScript, se evalúa un parámetro predeterminado cada vez que se llama a la función sin el parámetro respectivo.

En el ejemplo anterior, `anotherFunction()` no será llamado en absoluto si se provee el parámetro `text`.

Por otro lado, se llamará independientemente cada vez que `text` se omita.

Parámetros predeterminados en viejo código JavaScript

Años atrás, JavaScript no soportaba la sintaxis para parámetros predeterminados. Entonces se usaban otras formas para especificarlos.

En estos días, aún podemos encontrarlos en viejos scripts.

Por ejemplo, una verificación explícita de `undefined`:

```
1 function showMessage(from, text) {
2   if (text === undefined) {
3     text = 'sin texto dado';
4   }
5
6   alert( from + ": " + text );
7 }
```

... O usando el operador `||`:

```
function showMessage(from, text) {
  // Si el valor de "text" es falso, asignar el
  // esto asume que text == "" es lo mismo que s
  text = text || 'sin texto dado';
  ...
}
```

Parámetros predeterminados alternativos

A veces tiene sentido asignar valores predeterminados a los parámetros más tarde, después de la declaración de función.

Podemos verificar si un parámetro es pasado durante la ejecución de la función comparándolo con `undefined`:

```
function showMessage(text) {
  // ...

  if (text === undefined) { // si falta el parámetro
    text = 'mensaje vacío';
  }

  alert(text);
}

showMessage(); // mensaje vacío
```

...O podemos usar el operador `||`:

```
function showMessage(text) {
  // si text es indefinida o falsa, la establece a 'va
  text = text || 'vacío';
  ...
}
```

Los intérpretes de JavaScript modernos soportan el operador nullish coalescing `??`, que es mejor cuando el valor de `0` debe ser considerado "normal" en lugar de falso:

```
function showCount(count) {
  // si count es undefined o null, muestra "desconocid
  alert(count ?? "desconocido");
}

showCount(0); // 0
showCount(null); // desconocido
showCount(); // desconocido
```

Devolviendo un valor

Una función puede devolver un valor al código de llamada como resultado.

El ejemplo más simple sería una función que suma dos valores:

```
function sum(a, b) {
  return a + b;
}

let result = sum(1, 2);
alert( result ); // 3
```

La directiva `return` puede estar en cualquier lugar de la función. Cuando la ejecución lo alcanza, la función se detiene y el valor se devuelve al código de llamada (asignado al `result` anterior).

Puede haber muchos `return` en una sola función. Por ejemplo:

```
function checkAge(age) {
  if (age > 18) {
    return true;
  } else {
    return confirm('¿Tienes permiso de tus padres?');
  }
}

let age = prompt('¿Qué edad tienes?', 18);

if ( checkAge(age) ) {
  alert( 'Acceso otorgado' );
} else {
  alert( 'Acceso denegado' );
}
```

Es posible utilizar `return` sin ningún valor. Eso hace que la función salga o termine inmediatamente.

Por ejemplo:

```
function showMovie(age) {
  if ( !checkAge(age) ) {
    return;
  }

  alert( "Mostrándote la película" ); // (*)
  // ...
}
```

En el código de arriba, si `checkAge(age)` devuelve `false`, entonces `showMovie` no mostrará la `alert`.

Una función con un `return` vacío, o sin return, devuelve `undefined`

Si una función no devuelve un valor, es lo mismo que si devolviera `undefined`:

```
function doNothing() { /* empty */ }

alert( doNothing() === undefined ); // true
```

Un `return` vacío también es lo mismo que `return undefined`:

```
function doNothing() {
  return;
}

alert( doNothing() === undefined ); // true
```

Nunca agregue una nueva línea entre `return` y el valor

Para una expresión larga de `return`, puede ser tentador ponerlo en una línea separada, como esta:

```
return
 (una + expresion + o + cualquier + cosa * f(a)
```

Eso no funciona, porque JavaScript asume un punto y coma después del `return`. Eso funcionará igual que:

```
return;
 (una + expresion + o + cualquier + cosa * f(a)
```

Entonces, efectivamente se convierte en un return vacío. Deberíamos poner el valor en la misma línea.

Nomenclatura de funciones

Las funciones son acciones. Entonces su nombre suele ser un verbo. Debe ser breve, lo más preciso posible y describir lo que hace la función, para que alguien que lea el código obtenga una indicación de lo que hace la función.

Es una práctica generalizada comenzar una función con un prefijo verbal que describe vagamente la acción. Debe haber un acuerdo dentro del equipo sobre el significado de los prefijos.

Por ejemplo, funciones que comienzan con `"show"` usualmente muestran algo.

Funciones que comienza con...

- `"get…"` – devuelven un valor,
- `"calc…"` – calculan algo,
- `"create…"` – crean algo,
- `"check…"` – revisan algo y devuelven un boolean, etc.

Ejemplos de este tipo de nombres:

```
showMessage(..)     // muestra un mensaje
getAge(..)          // devuelve la edad (la obtiene de
calcSum(..)         // calcula una suma y devuelve el
createForm(..)      // crea un formulario (y usualment
checkPermission(..) // revisa permisos, y devuelve tru
```

Con los prefijos en su lugar, un vistazo al nombre de una función permite comprender qué tipo de trabajo realiza y qué tipo de valor devuelve.

Una función – una acción

Una función debe hacer exactamente lo que sugiere su nombre, no más.

Dos acciones independientes por lo general merecen dos funciones, incluso si generalmente se convocan juntas (en ese caso, podemos hacer una tercera función que llame a esas dos).

Algunos ejemplos de cómo se rompen estas reglas:

- `getAge` – está mal que muestre una `alert` con la edad (solo debe obtenerla).
- `createForm` – está mal que modifique el documento agregándole el form (solo debe crearlo y devolverlo).
- `checkPermission` – está mal que muestre el mensaje `acceso otorgado/denegado` (solo debe realizar la verificación y devolver el resultado).

En estos ejemplos asumimos los significados comunes de los prefijos. Tú y tu equipo pueden acordar significados diferentes, aunque usualmente no muy diferente. En cualquier caso, debe haber una compromiso firme de lo que significa un prefijo, de lo que una función con prefijo puede y no puede hacer. Todas las funciones con el mismo prefijo deben obedecer las reglas. Y el equipo debe compartir ese conocimiento.

Nombres de funciones ultracortos

Las funciones que se utilizan *muy a menudo* algunas veces tienen nombres ultracortos.

Por ejemplo, el framework jQuery define una función con `$`. La librería LoDash tiene como nombre de función principal `_`.

Estas son excepciones. En general, los nombres de las funciones deben ser concisos y descriptivos.

Funciones == Comentarios

Las funciones deben ser cortas y hacer exactamente una cosa. Si esa cosa es grande, tal vez valga la pena dividir la función en algunas funciones más pequeñas. A veces, seguir esta regla puede no ser tan fácil, pero definitivamente es algo bueno.

Una función separada no solo es más fácil de probar y depurar, – ¡su existencia es un gran comentario!

Por ejemplo, comparemos las dos funciones `showPrimes(n)` siguientes. Cada una devuelve números primos hasta `n`.

La primera variante usa una etiqueta:

```
function showPrimes(n) {
  nextPrime: for (let i = 2; i < n; i++) {

    for (let j = 2; j < i; j++) {
      if (i % j == 0) continue nextPrime;
    }

    alert( i ); // un número primo
  }
}
```

La segunda variante usa una función adicional `isPrime(n)` para probar la primalidad:

```
function showPrimes(n) {

  for (let i = 2; i < n; i++) {
    if (!isPrime(i)) continue;

    alert(i);  // a prime
  }
}

function isPrime(n) {
  for (let i = 2; i < n; i++) {
    if ( n % i == 0) return false;
  }
  return true;
}
```

La segunda variante es más fácil de entender, ¿no? En lugar del código, vemos un nombre de la acción. (`isPrime`). A veces las personas se refieren a dicho código como *autodescriptivo*.

Por lo tanto, las funciones se pueden crear incluso si no tenemos la intención de reutilizarlas. Estructuran el código y lo hacen legible.

Resumen

Una declaración de función se ve así:

```
function name(parámetros, delimitados, por, coma) {
  /* code */
}
```

- Los valores pasados a una función como parámetros se copian a sus variables locales.
- Una función puede acceder a variables externas. Pero funciona solo de adentro hacia afuera. El código fuera de la función no ve sus variables locales.
- Una función puede devolver un valor. Si no lo hace, entonces su resultado es `undefined`.

Para que el código sea limpio y fácil de entender, se recomienda utilizar principalmente variables y parámetros locales en la función, no variables externas.

Siempre es más fácil entender una función que obtiene parámetros, trabaja con ellos y devuelve un resultado que una función que no obtiene parámetros, pero modifica las variables externas como un efecto secundario.

Nomenclatura de funciones:

- Un nombre debe describir claramente lo que hace la función. Cuando vemos una llamada a la función en el código, un buen nombre nos da al instante una comprensión de lo que hace y devuelve.
- Una función es una acción, por lo que los nombres de las funciones suelen ser verbales.
- Existen muchos prefijos de funciones bien conocidos como `create…`, `show…`, `get…`, `check…` y así. Úsalos para insinuar lo que hace una función.

Las funciones son los principales bloques de construcción de los scripts. Ahora hemos cubierto los conceptos básicos, por lo que en realidad podemos comenzar a crearlos y usarlos. Pero ese es solo el comienzo del camino. Volveremos a ellos muchas veces, profundizando en sus funciones avanzadas.

Tareas

¿Es "else" requerido?

importancia: 4

La siguiente función devuelve `true` si el parámetro `age` es mayor a `18`.

De lo contrario, solicita una confirmación y devuelve su resultado:

```
function checkAge(age) {
  if (age > 18) {
    return true;
```

```
  } else {
    // ...
    return confirm('¿Tus padres te permitieron?');
  }
}
```

¿Funcionará la función de manera diferente si se borra `else`?

```
function checkAge(age) {
  if (age > 18) {
    return true;
  }
  // ...
  return confirm('¿Tus padres te permitieron?');
}
```

¿Hay alguna diferencia en el comportamiento de estas dos variantes?

solución

Reescribe la función utilizando '?' o '||'

importancia: 4

La siguiente función devuelve `true` si el parámetro `age` es mayor que `18`.

De lo contrario, solicita una confirmación y devuelve su resultado.

```
function checkAge(age) {
  if (age > 18) {
    return true;
  } else {
    return confirm('¿Tienes permiso de tus padres?');
  }
}
```

Reescríbela para realizar lo mismo, pero sin `if`, en una sola linea.

Haz dos variantes de `checkAge`:

1. Usando un operador de signo de interrogación `?`
2. Usando OR `||`

solución

Función min(a, b)

importancia: 1

Escriba una función `min(a,b)` la cual devuelva el menor de dos números `a` y `b`.

Por ejemplo:

```
min(2, 5) == 2
min(3, -1) == -1
min(1, 1) == 1
```

solución

Función pow(x,n)

importancia: 4

Escriba la función `pow(x,n)` que devuelva `x` como potencia de `n`. O, en otras palabras, multiplique `x` por si mismo `n` veces y devuelva el resultado.

```
pow(3, 2) = 3 * 3 = 9
pow(3, 3) = 3 * 3 * 3 = 27
pow(1, 100) = 1 * 1 * ...* 1 = 1
```

Cree una página web que solicite `x` y `n`, y luego muestre el resultado de `pow(x,n)`.

Ejecutar el demo

PD: En esta tarea, la función solo debe admitir valores naturales de `n`: enteros desde `1`.

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor acerca del proyecto contáctenos

14 de febrero de 2025

Expresiones de función

En JavaScript, una función no es una "estructura mágica del lenguaje", sino un tipo de valor especial.

La sintaxis que usamos antes se llama *Declaración de Función*:

```
function sayHi() {
  alert( "Hola" );
}
```

Existe otra sintaxis para crear una función que se llama una *Expresión de Función*.

Esto nos permite crear una nueva función en el medio de cualquier expresión

Por ejemplo:

```
let sayHi = function() {
  alert( "Hola" );
};
```

Aquí podemos ver una variable `sayHi` obteniendo un valor —la nueva función — creada como `function() { alert("Hello"); }`.

Como la creación de una función ocurre en el contexto de una expresión de asignación, (el lado derecho de `=`), esto es una *Expresión de función*.

Note que no hay un nombre después de la palabra clave `function`. Omitir el nombre está permitido en las expresiones de función.

Aquí la asignamos directamente a la variable, así que el significado de estos ejemplos de código es el mismo: "crear una función y ponerla en la variable `sayHi`".

En situaciones más avanzadas, que cubriremos más adelante, una función puede ser creada e inmediatamente llamada o agendada para uso posterior, sin almacenarla en ningún lugar, permaneciendo así anónima.

La función es un valor

Reiteremos: no importa cómo es creada la función, una función es un valor. Ambos ejemplos arriba almacenan una función en la variable `sayHi`.

Incluso podemos mostrar aquel valor usando `alert`:

```
function sayHi() {
  alert( "Hola" );
}


```

```
alert( sayHi ); // muestra el código de la función
```

Tenga en cuenta que la última línea no ejecuta la función, porque no hay paréntesis después de `sayHi`. Existen lenguajes de programación en los que cualquier mención del nombre de una función causa su ejecución, pero JavaScript no funciona así.

En JavaScript, una función es un valor, por lo tanto podemos tratarlo como un valor. El código de arriba muestra su representación de cadena, que es el código fuente.

Por supuesto que es un valor especial, en el sentido que podemos invocarlo de esta forma `sayHi()`.

Pero sigue siendo un valor. Entonces podemos trabajar con ello como trabajamos con otro tipo de valores.

Podemos copiar una función a otra variable:

```
function sayHi() {   // (1) crear
  alert( "Hola" );
}

let func = sayHi;    // (2) copiar

func(); // Hola          // (3) ejecuta la copia (func
sayHi(); // Hola         // esto también funciona (por
```

Esto es lo que sucede arriba en detalle:

1. La Declaración de la Función `(1)` crea la función y la coloca dentro de la variable llamada `sayHi`.
2. Línea `(2)` copia la función en la variable `func`.
3. Ahora la función puede ser llamada de ambas maneras, `sayHi()` y `func()`.

También podríamos haber usado una expresión de función para declarar `sayHi` en la primera línea:

```
let sayHi = function() { // (1) crea
  alert( "Hola" );
};

let func = sayHi;  //(2)
// ...
```

Todo funcionaría igual.

¿Por qué hay un punto y coma al final?

Tal vez te preguntes por qué la Expresión de Función tiene un punto y coma `;` al final, pero la Declaración de Función no lo tiene:

```
function sayHi() {
  // ...
}

let sayHi = function() {
  // ...
};
```

La respuesta es simple: una expresión de función se crea aquí como `function(…) {…}` dentro de la sentencia de asignación `let sayHi = …;` . El punto y coma se recomienda para finalizar la sentencia, no es parte de la sintaxis de función.

El punto y coma estaría allí para una asignación más simple tal como `let sayHi = 5;` , y también está allí para la asignación de función.

Funciones Callback

Veamos más ejemplos del pasaje de funciones como valores y el uso de expresiones de función.

Escribimos una función `ask(question, yes, no)` con tres argumentos:

`question`

Texto de la pregunta

`yes`

Función a ejecutar si la respuesta es “Yes”

`no`

Función a ejecutar si la respuesta es “No”

La función deberá preguntar la `question` y, dependiendo de la respuesta del usuario, llamar `yes()` o `no()` :

```
function ask(question, yes, no) {
  if (confirm(question)) yes()
  else no();
}

function showOk() {
  alert( "Estás de acuerdo." );
}

function showCancel() {
  alert( "Cancelaste la ejecución." );
}

// uso: las funciones showOk, showCancel son pasadas c
ask("Estás de acuerdo?", showOk, showCancel);
```

En la práctica, tales funciones son bastante útiles. La mayor diferencia entre la función `ask` en la vida real y el ejemplo anterior es que las funciones de la vida

real utilizan formas para interactuar con el usuario más complejas que un simple `confirme`. En el navegador, una función como tal normalmente dibuja una ventana de pregunta atractiva. Pero esa es otra historia.

Los argumentos de `ask` se llaman *funciones callback* o simplemente *callbacks*.

La idea es que pasamos una función y esperamos que se "devuelva la llamada" más tarde si es necesario. En nuestro caso, `showOk` se convierte en la callback para la respuesta "Yes", y `showCancel` para la respuesta "No".

Podemos usar Expresión de Función para redactar una función equivalente y más corta:

```
function ask(question, yes, no) {
  if (confirm(question)) yes()
  else no();
}

ask(
  "Estás de acuerdo?",
  function() { alert("Estás de acuerdo"); },
  function() { alert("Cancelaste la ejecución."); }
);
```

Aquí, las funciones son declaradas justo dentro del llamado `ask(...)`. No tienen nombre, y por lo tanto se denominan *anónimas*. Tales funciones no se pueden acceder fuera de `ask` (porque no están asignadas a variables), pero eso es justo lo que queremos aquí.

Éste código aparece en nuestros scripts de manera muy natural, está en el archivo de comandos de JavaScript.

> **Una función es un valor representando una "acción"**
>
> Valores regulares tales como cadena de caracteres o números representan los *datos*.
>
> Una función puede ser percibida como una *acción*.
>
> La podemos pasar entre variables y ejecutarla cuando nosotros queramos.

Expresión de Función vs Declaración de Función

Formulemos las principales diferencias entre Declaración y Expresión de Funciones.

Primero, la sintaxis: cómo diferenciarlas en el código.

- *Declaración de Función:* una función, declarada como una instrucción separada, en el flujo de código principal.

```
// Declaración de Función
function sum(a, b) {

```

```
  return a + b;
}
```

- *Expresión de Función:* una función, creada dentro de una expresión o dentro de otra construcción sintáctica. Aquí, la función es creada en el lado derecho de la "expresión de asignación" `=`:

```
// Expresión de Función
let sum = function(a, b) {
  return a + b;
};
```

La diferencia más sutil es *cuándo* la función es creada por el motor de JavaScript.

Una Expresión de Función es creada cuando la ejecución la alcance y es utilizable desde ahí en adelante.

Una vez que el flujo de ejecución pase al lado derecho de la asignación `let sum = function…` – aquí vamos, la función es creada y puede ser usada (asignada, llamada, etc.) de ahora en adelante.

Las Declaraciones de Función son diferente.

Una Declaración de Función puede ser llamada antes de ser definida.

Por ejemplo, una Declaración de Función global es visible en todo el script, sin importar dónde se esté.

Esto se debe a los algoritmos internos. Cuando JavaScript se prepara para ejecutar el script, primero busca Declaraciones de Funciones globales en él y crea las funciones. Podemos pensar en esto como una "etapa de inicialización".

Y después de que se procesen todas las Declaraciones de Funciones, el código se ejecuta. Entonces tiene acceso a éstas funciones.

Por ejemplo, esto funciona:

```
sayHi("John"); // Hola, John

function sayHi(name) {
  alert( `Hola, ${name}` );
}
```

La Declaración de Función `sayHi` es creada cuando JavaScript está preparándose para iniciar el script y es visible en todas partes.

...Si fuera una Expresión de Función, entonces no funcionaría:

```
sayHi("John"); // error!

let sayHi = function(name) {  // (*) ya no hay magia
  alert( `Hola, ${name}` );
};
```

Las Expresiones de Función son creadas cuando la ejecución las alcance. Esto podría pasar solamente en la línea `(*)`. Demasiado tarde.

Otra característica especial de las Declaraciones de Funciones es su alcance de bloque.

En modo estricto, cuando una Declaración de Función se encuentra dentro de un bloque de código, es visible en todas partes dentro de ese bloque. Pero no fuera de él.

Por ejemplo, imaginemos que necesitamos declarar una función `welcome()` dependiendo de la variable `age` que obtengamos durante el tiempo de ejecución. Y luego planeamos usarlo algún tiempo después.

Si utilizamos la Declaración de Funciones, no funcionará como se esperaba:

```
let age = prompt("Cuál es tu edad?", 18);

// declarar condicionalmente una función
if (age < 18) {

  function welcome() {
    alert("Hola!");
  }

} else {

  function welcome() {
    alert("Saludos!");
  }

}

// ...usarla más tarde
welcome(); // Error: welcome no está definida
```

Esto se debe a que una Declaración de Función sólo es visible dentro del bloque de código en el que reside.

Aquí hay otro ejemplo:

```
let age = 16; // tomemos 16 como ejemplo

if (age < 18) {
  welcome();               // \   (corre)
                           //  |
  function welcome() {     //  |
    alert("¡Hola!");       //  |  La declaración de Fu
  }                        //  |  en todas partes del
                           //  |
  welcome();               // /   (corre)

} else {

  function welcome() {
    alert("¡Saludos!");
  }
}

// Aquí estamos fuera de las llaves,
// por lo tanto no podemos ver la Declaración de Funci

```

```
welcome(); // Error: welcome no está definida
```

¿Qué podemos hacer para que `welcome` sea visible fuera de 'if'?

El enfoque correcto sería utilizar una Expresión de Función y asignar `welcome` a la variable que se declara fuera de 'if' y tiene la visibilidad adecuada.

Este código funciona según lo previsto:

```
let age = prompt("Cuál es tu edad?", 18);

let welcome;

if (age < 18) {

  welcome = function() {
    alert("Hola!");
  };

} else {

  welcome = function() {
    alert("Saludos!");
  };

}

welcome(); // ahora ok
```

O lo podemos simplificar aun más usando un operador de signo de pregunta `?`:

```
let age = prompt("¿Cuál es tu edad?", 18);

let welcome = (age < 18) ?
  function() { alert("¡Hola!"); } :
  function() { alert("¡Saludos!"); };

welcome(); // ahora ok
```

¿Cuándo debo elegir la Declaración de Función frente a la Expresión de Función?

Como regla general, cuando necesitamos declarar una función, la primera que debemos considerar es la sintaxis de la Declaración de Función. Da más libertad en cómo organizar nuestro código, porque podemos llamar a tales funciones antes de que sean declaradas.

También es un poco más fácil de buscar `function f(…) {…}` en el código comparado con `let f = function(…) {…}`. La Declaración de Función es más llamativa.

...Pero si una Declaración de Función no nos conviene por alguna razón, o necesitamos declaración condicional (hemos visto un ejemplo),

entonces se debe usar la Expresión de función.

Resumen

- Las funciones son valores. Se pueden asignar, copiar o declarar en cualquier lugar del código.
- Si la función se declara como una declaración separada en el flujo del código principal, eso se llama "Declaración de función".
- Si la función se crea como parte de una expresión, se llama "Expresión de función".
- Las Declaraciones de Funciones se procesan antes de ejecutar el bloque de código. Son visibles en todas partes del bloque.
- Las Expresiones de Función se crean cuando el flujo de ejecución las alcanza.

En la mayoría de los casos, cuando necesitamos declarar una función, es preferible una Declaración de Función, ya que es visible antes de la declaración misma. Eso nos da más flexibilidad en la organización del código, y generalmente es más legible.

Por lo tanto, deberíamos usar una Expresión de Función solo cuando una Declaración de Función no sea adecuada para la tarea. Hemos visto un par de ejemplos de eso en este capítulo, y veremos más en el futuro.

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

3 de julio de 2022

Funciones Flecha, lo básico

Hay otra sintaxis muy simple y concisa para crear funciones, que a menudo es mejor que las Expresiones de funciones.

Se llama "funciones de flecha", porque se ve así:

```
let func = (arg1, arg2, ..., argN) => expression;
```

Esto crea una función `func` que acepta los parámetros `arg1..argN`, luego evalúa la `expression` del lado derecho mediante su uso y devuelve su resultado.

En otras palabras, es la versión más corta de:

```
let func = function(arg1, arg2, ..., argN) {
  return expression;
};
```

Veamos un ejemplo concreto:

```
let sum = (a, b) => a + b;

/* Esta función de flecha es una forma más corta de:

let sum = function(a, b) {
  return a + b;
};
*/

alert( sum(1, 2) ); // 3
```

Como puedes ver, `(a, b) => a + b` significa una función que acepta dos argumentos llamados `a` y `b`. Tras la ejecución, evalúa la expresión `a + b` y devuelve el resultado.

- Si solo tenemos un argumento, se pueden omitir paréntesis alrededor de los parámetros, lo que lo hace aún más corto.

  Por ejemplo:

```
let double = n => n * 2;
// Más o menos lo mismo que: let double = function(n
alert( double(3) ); // 6
```

- Si no hay parámetros, los paréntesis estarán vacíos; pero deben estar presentes:

```
let sayHi = () => alert("¡Hola!");

sayHi();
```

Las funciones de flecha se pueden usar de la misma manera que las expresiones de función.

Por ejemplo, para crear dinámicamente una función:

```
let age = prompt("What is your age?", 18);

let welcome = (age < 18) ?
  () => alert('¡Hola!') :
  () => alert("¡Saludos!");

welcome();
```

Las funciones de flecha pueden parecer desconocidas y poco legibles al principio, pero eso cambia rápidamente a medida que los ojos se acostumbran a la estructura.

Son muy convenientes para acciones simples de una línea, cuando somos demasiado flojos para escribir muchas palabras.

Funciones de flecha multilínea

Las funciones de flecha que estuvimos viendo eran muy simples. Toman los parámetros a la izquierda de `=>`, los evalúan y devuelven la expresión del lado derecho.

A veces necesitamos una función más compleja, con múltiples expresiones o sentencias. En ese caso debemos encerrarlos entre llaves. La diferencia principal es que las llaves necesitan usar un `return` para devolver un valor (tal como lo hacen las funciones comunes).

Como esto:

```
let sum = (a, b) => {  // la llave abre una función mu
  let result = a + b;
  return result; // si usamos llaves, entonces necesit
};

alert( sum(1, 2) ); // 3
```

Más por venir

Aquí elogiamos las funciones de flecha por su brevedad. ¡Pero eso no es todo!

Las funciones de flecha tienen otras características interesantes.

Para estudiarlas en profundidad, primero debemos conocer algunos otros aspectos de JavaScript, por lo que volveremos a las funciones de flecha más adelante en el capítulo Funciones de flecha revisadas.

Por ahora, ya podemos usar las funciones de flecha para acciones de una línea y devoluciones de llamada.

Resumen

Las funciones de flecha son útiles para acciones simples, especialmente las de una sola línea. Vienen en dos variantes:

1. Sin llaves: `(...args) => expression` – el lado derecho es una expresión: la función la evalúa y devuelve el resultado. Pueden omitirse los paréntesis si solo hay un argumento, por ejemplo `n => n*2`.
2. Con llaves: `(...args) => { body }` – las llaves nos permiten escribir varias declaraciones dentro de la función, pero necesitamos un `return` explícito para devolver algo.

Tareas

Reescribe con funciones de flecha

Reemplace las expresiones de función con funciones de flecha en el código a continuación:

```
function ask(question, yes, no) {
  if (confirm(question)) yes();
  else no();
}

ask(
  "Do you agree?",
  function() { alert("You agreed."); },
  function() { alert("You canceled the execution."); }
);
```

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor acerca del proyecto

contáctenos

13 de junio de 2022

Especiales JavaScript

Este capítulo resume brevemente las características de JavaScript que hemos aprendido hasta ahora, prestando especial atención a los detalles relevantes.

Estructura de Código

Las declaraciones se delimitan con un punto y coma:

```
alert('Hola'); alert('Mundo');
```

En general, un salto de línea también se trata como un delimitador, por lo que también funciona:

```
alert('Hola')
alert('Mundo')
```

Esto se llama "inserción automática de punto y coma". A veces no funciona, por ejemplo:

```
alert("Habrá un error después de este mensaje.")

[1, 2].forEach(alert)
```

La mayoría de las guías de estilo de código coinciden en que debemos poner un punto y coma después de cada declaración.

Los puntos y comas no son necesarios después de los bloques de código `{...}` y los constructores de sintaxis como los bucles:

```
function f() {
  // no se necesita punto y coma después de la declara
}

for(;;) {
  // no se necesita punto y coma después del bucle
}
```

...Pero incluso si colocásemos un punto y coma "extra" en alguna parte, eso no sería un error. Solo sería ignorado.

Más en: Estructura del código.

Modo estricto

Para habilitar completamente todas las características de JavaScript moderno, debemos comenzar los scripts con `"use strict"`.

```
'use strict';

...
```

La directiva debe estar en la parte superior de un script o al comienzo de una función.

Sin la directiva `"use strict"` todo sigue funcionando, pero algunas características se comportan de la manera antigua y "compatible". Generalmente preferimos el comportamiento moderno.

Algunas características modernas del lenguaje (como las clases que estudiaremos en el futuro) activan el modo estricto implícitamente.

Más en: El modo moderno, "use strict".

Variables

Se pueden declarar usando:

- `let`
- `const` (constante, no se puede cambiar)
- `var` (estilo antiguo, lo veremos más tarde)

Un nombre de variable puede incluir:

- Letras y dígitos, pero el primer carácter no puede ser un dígito.
- Los caracteres `$` y `_` son normales, al igual que las letras.
- Los alfabetos y jeroglíficos no latinos también están permitidos, pero comúnmente no se usan.

Las variables se escriben dinámicamente. Pueden almacenar cualquier valor:

```
let x = 5;
x = "John";
```

Hay 8 tipos de datos:

- `number` tanto para números de punto flotante como enteros,
- `bigint` para números enteros de largo arbitrario,
- `string` para textos,
- `boolean` para valores lógicos: `true/false`,
- `null` – un tipo con el valor único `null`, que significa "vacío" o "no existe",
- `undefined` – un tipo con el valor único `undefined`, que significa "no asignado",
- `object` y `symbol` – para estructuras de datos complejas e identificadores únicos, aún no los hemos aprendido.

El operador `typeof` devuelve el tipo de un valor, con dos excepciones:

```
typeof null == "object" // error del lenguaje
typeof function(){} == "function" // las funciones son
```

Más en: Variables y Tipos de datos.

Interacción

Estamos utilizando un navegador como entorno de trabajo, por lo que las funciones básicas de la interfaz de usuario serán:

`prompt(question, [default])`

Hace una pregunta `question`, y devuelve lo que ingresó el visitante o `null` si presiona "cancelar".

`confirm(question)`

Hace una pregunta `question`, y sugiere elegir entre Aceptar y Cancelar. La elección se devuelve como booleano `true/false`.

`alert(message)`

Muestra un `message`.

Todas estas funciones son *modales*, pausan la ejecución del código y evitan que el visitante interactúe con la página hasta que responda.

Por ejemplo:

```
let userName = prompt("¿Su nombre?", "Alice");
let isTeaWanted = confirm("¿Quiere té?");

alert( "Visitante: " + userName ); // Alice
alert( "Quiere té: " + isTeaWanted ); // true
```

Más en: Interacción: alert, prompt, confirm.

Operadores

JavaScript soporta los siguientes operadores:

Aritméticos

Los normales: `* + - /`, también `%` para los restos y `**` para aplicar potencia de un número.

El binario más `+` concatena textos. Si uno de los operandos es un texto, el otro también se convierte en texto:

```
alert( '1' + 2 ); // '12', texto
alert( 1 + '2' ); // '12', texto
```

Asignaciones

Existen las asignaciones simples: `a = b` y las combinadas `a *= 2`.

Operador bit a bit

Los operadores bit a bit funcionan con enteros de 32 bits al más bajo nivel, el de bit: vea la documentación cuando los necesite.

Condicional

El único operador con 3 parámetros: `cond ? resultA : resultB`. Sí `cond` es verdadera, devuelve `resultA`, de lo contrario `resultB`.

Operadores Lógicos

Los operadores lógicos Y `&&` y Ó `||` realizan una evaluación de circuito corto y luego devuelven el valor donde se detuvo (no necesariamente true/false). El operador lógico NOT `!` convierte el operando a tipo booleano y devuelve el valor inverso.

Operador "Nullish coalescing"

El operador `??` brinda una forma de elegir el primer valor "definido" de una lista de variables. El resultado de `a ?? b` es `a` salvo que esta sea `null/undefined`, en cuyo caso será `b`.

Comparaciones

Para verificar la igualdad `==` de valores de diferentes tipos, estos se convierten a número (excepto `null` y `undefined` que son iguales entre sí y nada más), por lo que son iguales:

```
1 alert( 0 == false ); // true
2 alert( 0 == '' ); // true
```

Otras comparaciones también se convierten en un número.

El operador de igualdad estricta `===` no realiza la conversión: diferentes tipos siempre significan diferentes valores.

Los valores `null` y `undefined` son especiales: son iguales `==` el uno al otro y no son iguales a nada más.

Las comparaciones mayor/menor comparan las cadenas carácter por carácter, los demás tipos de datos se convierten a número.

Otros operadores

Hay algunos otros, como un operador de coma.

Más en: Operadores básicos, matemáticas, Comparaciones, Operadores Lógicos, Operador Nullish Coalescing '??'.

Bucles

- Cubrimos 3 tipos de bucles:

```
1 // 1
2 while (condition) {
3   ...
4 }
5
6 // 2
7 do {
8   ...
```

```
} while (condition);

// 3
for(let i = 0; i < 10; i++) {
  ...
}
```

- La variable declarada en el bucle `for(let...)` sólo es visible dentro del bucle. Pero también podemos omitir el `let` y reutilizar una variable existente.
- Directivas `break/continue` permiten salir de todo el ciclo/iteración actual. Use etiquetas para romper bucles anidados.

Detalles en: Bucles: while y for.

Más adelante estudiaremos más tipos de bucles para tratar con objetos.

La construcción “switch”

La construcción “switch” puede reemplazar múltiples revisiones con `if`. “switch” utiliza `===` (comparación estricta).

Por ejemplo:

```
let age = prompt('¿Su Edad?', 18);

switch (age) {
  case 18:

    alert("No funciona"); // el resultado de la petici

  case "18":
    alert("¡Funciona!");
    break;

  default:
    alert("Todo valor que no sea igual a uno de arriba
}
```

Detalles en: La sentencia "switch".

Funciones

Cubrimos tres formas de crear una función en JavaScript:

1. Declaración de función: la función en el flujo del código principal

```
function sum(a, b) {
  let result = a + b;

  return result;
}
```

2. Expresión de función: la función en el contexto de una expresión

```
let sum = function(a, b) {
  let result = a + b;

  return result;
};
```

3. Funciones de flecha:

```
// la expresión en el lado derecho
let sum = (a, b) => a + b;

// o sintaxis multilínea { ... }, aquí necesita retu
let sum = (a, b) => {
  // ...
  return a + b;
}

// sin argumentos
let sayHi = () => alert("Hello");

// con un único argumento
let double = n => n * 2;
```

- Las funciones pueden tener variables locales: son aquellas declaradas dentro de su cuerpo. Estas variables solo son visibles dentro de la función.
- Los parámetros pueden tener valores predeterminados: `function sum(a = 1, b = 2) {...}`.
- Las funciones siempre devuelven algo. Si no hay `return`, entonces el resultado es `undefined`.

Más: ver Funciones, Funciones Flecha, lo básico.

Más por venir

Esa fue una breve lista de características de JavaScript. Por ahora solo hemos estudiado lo básico. Más adelante en el tutorial encontrará más características especiales y avanzadas de JavaScript.

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor acerca del proyecto
contáctenos